



三维点云处理技术 和深度学习在点云处理中的应用

三维点云数据处理基础

索传哲



内容概要

点云滤波方法

点云组织形式与最近邻搜索

点云分割 拟合 聚类方法

激光雷达运动补偿方法

点云去地面原理及方法

Point Cloud Library

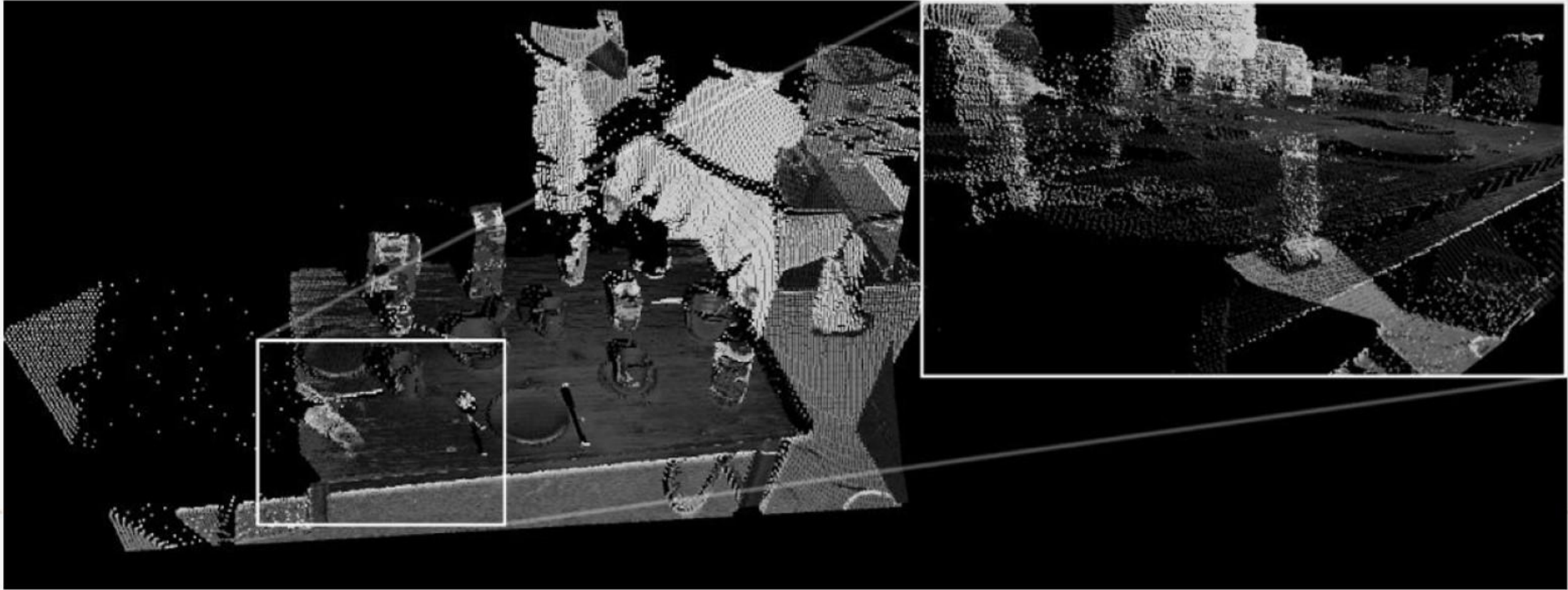


- ▶ 点云 = nD 点的 “云” (集合) (通常 $n = 3$)
- ▶ $p_i = \{x_i, y_i, z_i\} \rightarrow P = \{p_1, p_2, \dots, p_i, \dots, p_n\}$
- ▶ 表征世界三维信息的常用方法

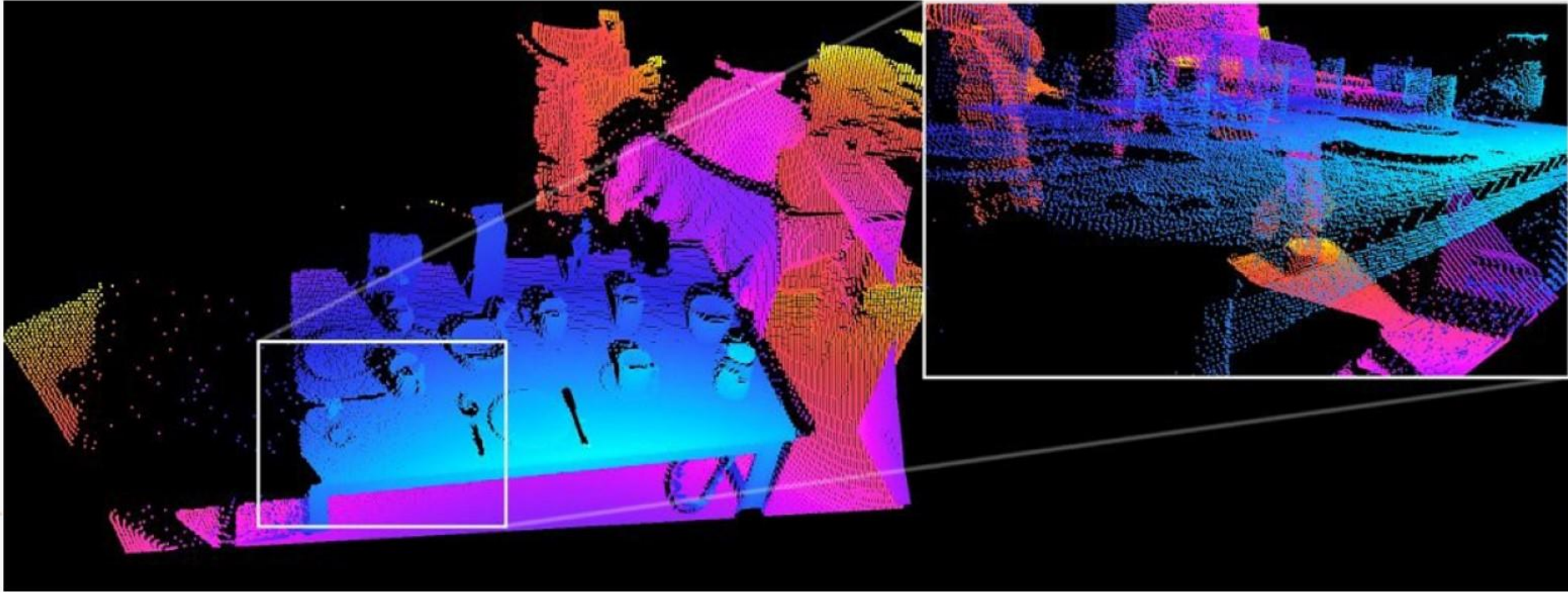


- ▶ 除了 XYZ 数据, 每个点 p_i 可以包含更多的信息
- ▶ 例如: RGB, 强度(intensity values), 距离(distances), 分割语义特征(segmentation results), 等

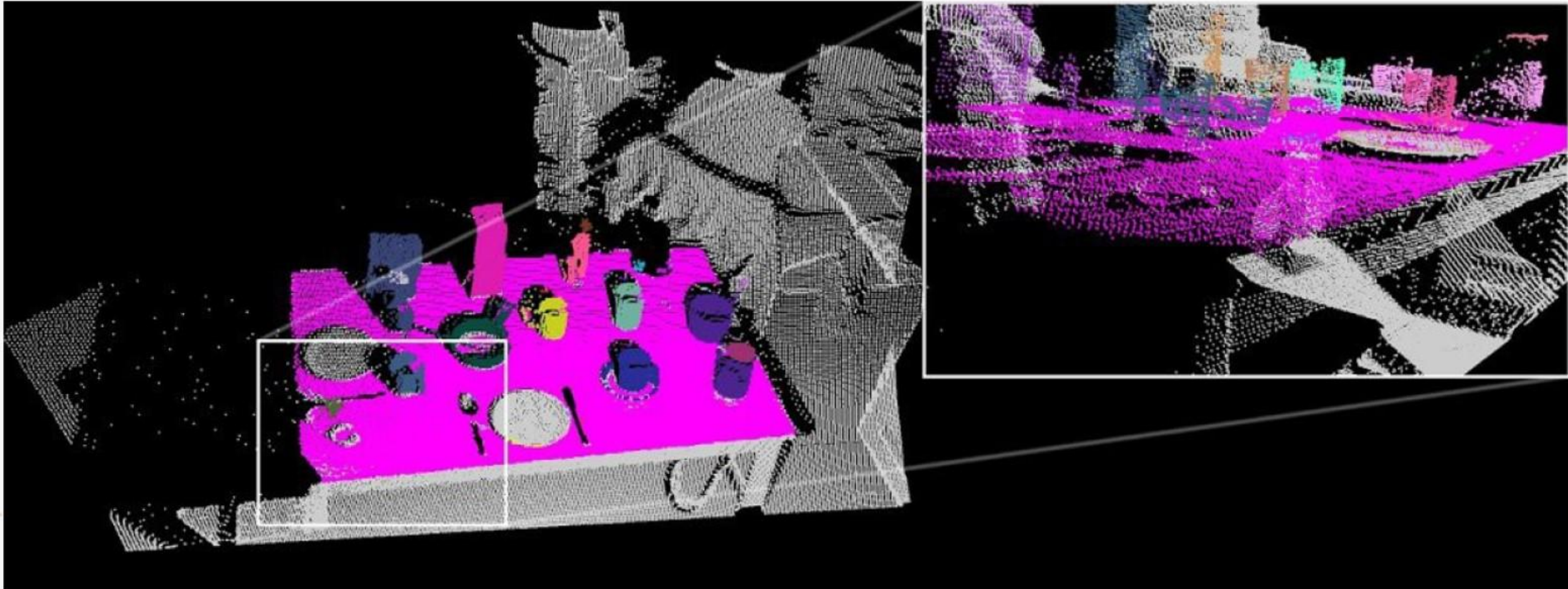
Point Cloud Library



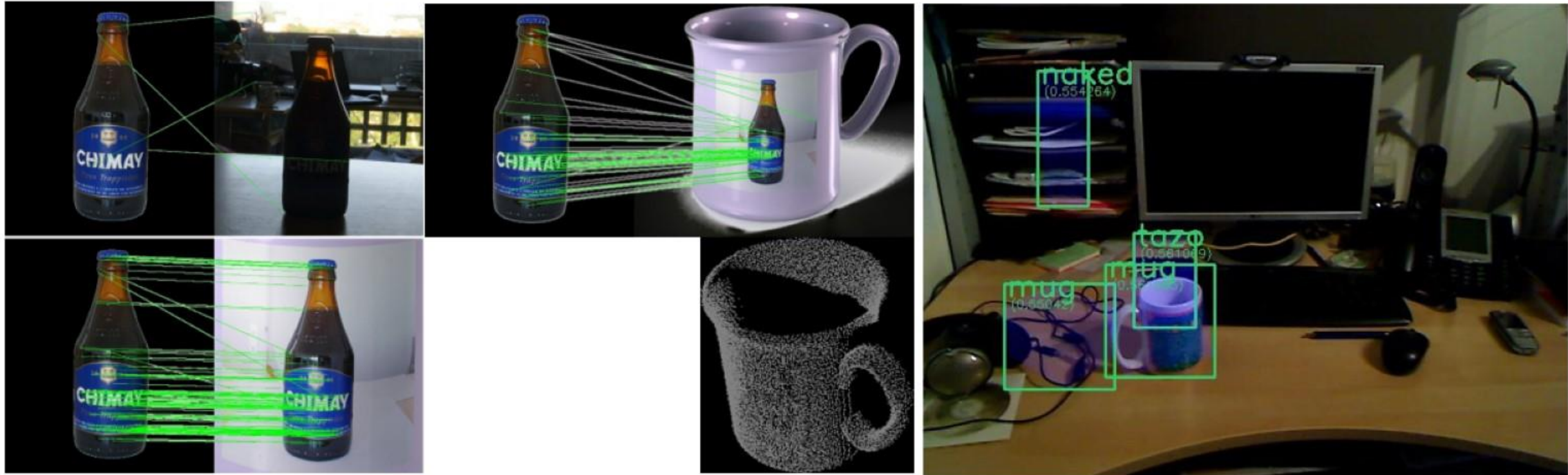
Point Cloud Library



Point Cloud Library

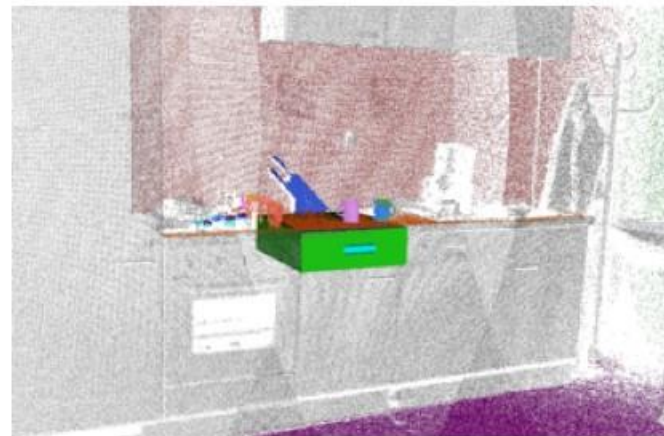
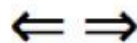


Point Cloud Library 3D Perception. 50% better.



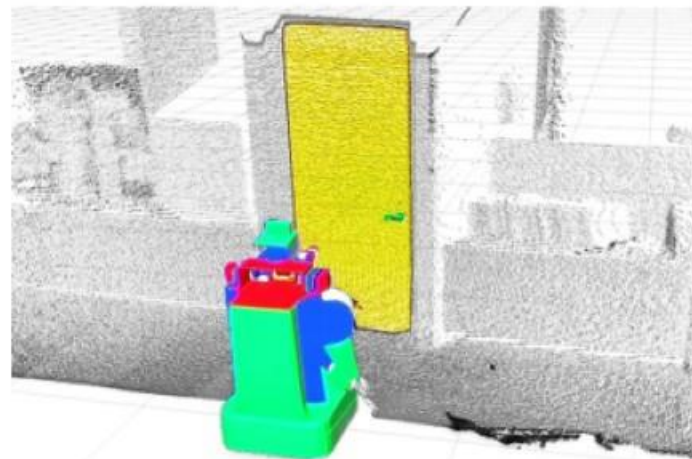
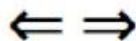
Point Cloud Library 3D Perception. 50% better.

➤ 应用案例：抽屉中取杯子



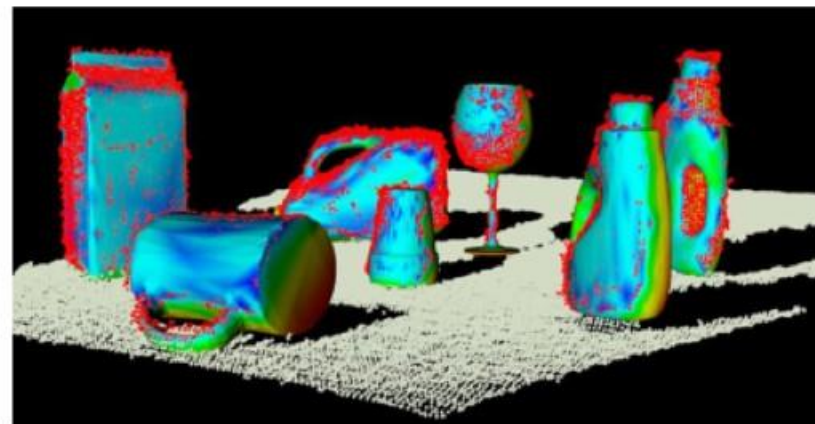
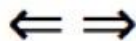
Point Cloud Library 3D Perception. 50% better.

➤ 应用案例：寻找门和门把手



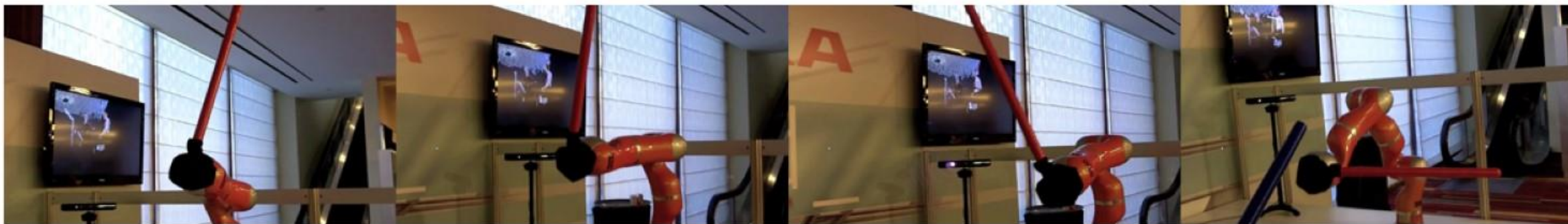
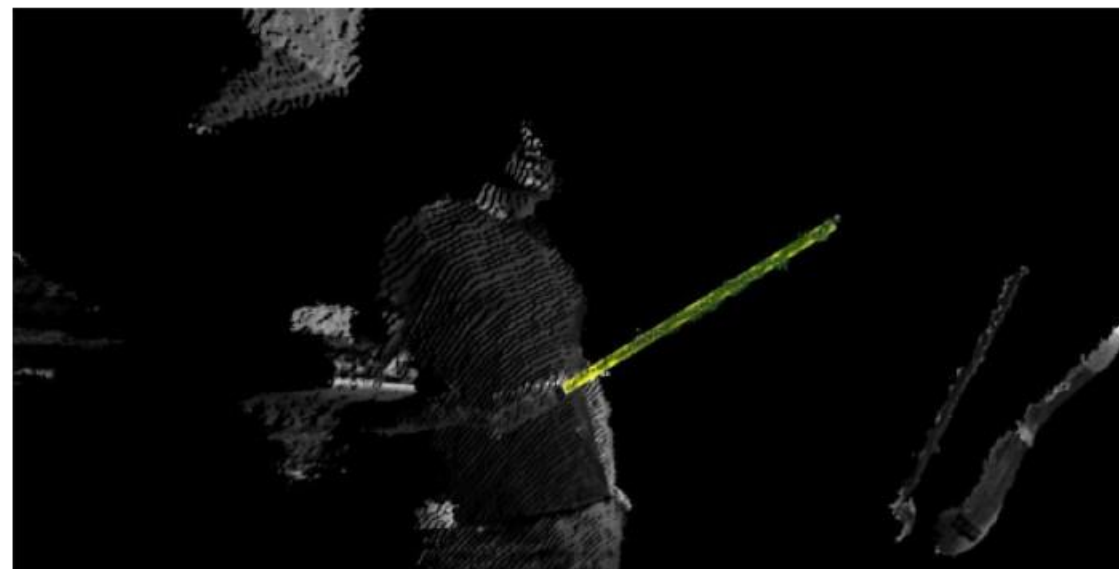
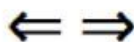
Point Cloud Library 3D Perception. 50% better.

➤ 应用案例：服务机器人物体抓取



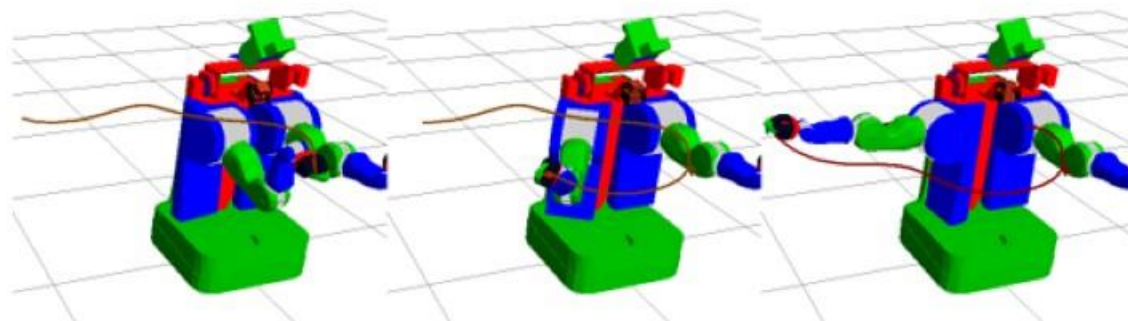
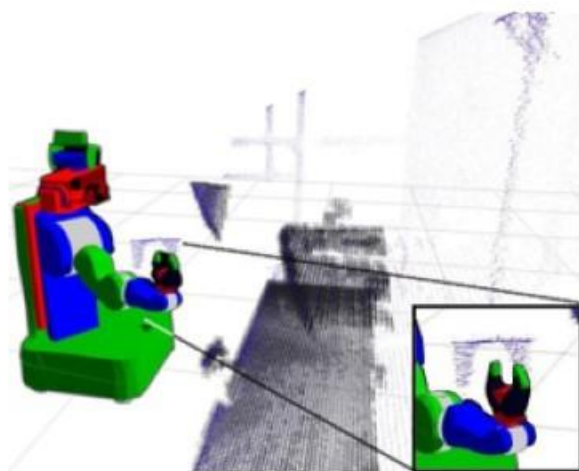
Point Cloud Library 3D Perception. 50% better.

➤ 应用案例: Fighting



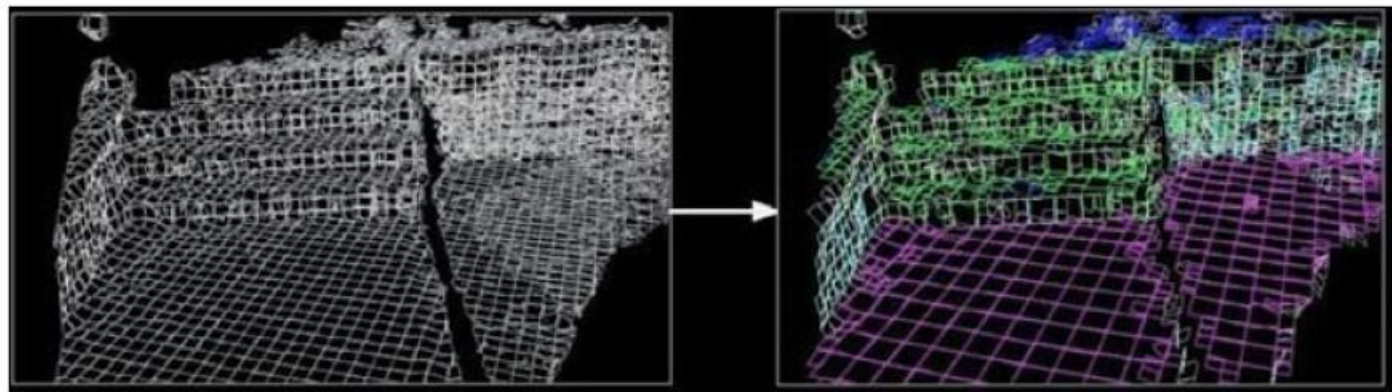
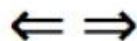
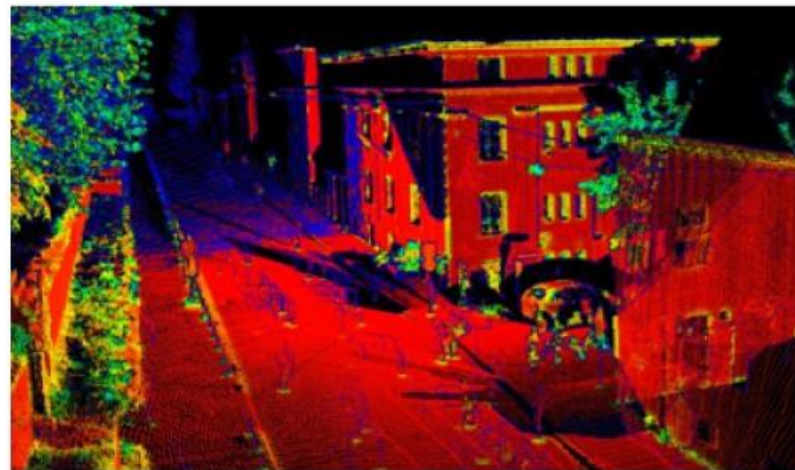
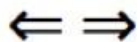
Point Cloud Library 3D Perception. 50% better.

➤ 应用案例：安全的运动规划与操作



Point Cloud Library 3D Perception. 50% better.

➤ 应用案例：自动驾驶与机器人



Point Cloud Library

➤ PCL点云数据结构

点云表示的数据结构:

- XYZ点的表示:

```
float32 x  
float32 y  
float32 z
```

- 或者N维点的表示:

```
float32[] point  
std::vector<float32> point
```

- **潜在问题**: 所有数据都要用 floats 类型 (!)
但是点的表示类型是多种多样的

点云数据通常很大:

- 操作运算通常较慢 (数据量大, 计算量大)
- 存储空间需求大

因此:

- 点云可以表示为:

```
PointT[] points
```

或者:

```
std::vector<PointT> points  
template<typename PointT>
```

- C++ PointT 可以为结构体(struct)或者通用数据类型

因此:

- 组合数据, 内存对齐满足SSE, 计算效率高
- 存储不同数据用不同的格式 (例如 rgb-24bits)

Point Cloud Library

➤ PCL点云数据结构

- `PointXYZI`的数据结构:

```
union{
float data[4];
struct
{
float x;
float y;
float z;
}; };
union{
struct
{
float intensity;
};
float data_c[4];
};
```

- 索引访问:

```
points[i].data[0]
points[i].x
points[i].intensity
```

- `Normal`的数据结构:

```
union
{
float data_n[4];
float normal[3];
struct
{
float normal_x;
float normal_y;
float normal_z;
}; };
union{
struct
{
float curvature;
};
float data_c[4];
};
```

- 索引访问:

```
points[i].data_n[0]
points[i].normal[0]
points[i].normal_x
```

- `PointXYZ`
- `PointXYZI`
- `PointXYZRGBA`
- `PointXYZRGB`
- `PointXY`
- `InterestPoint`
- `Normal`
- `PointNormal`
- `PointXYZRGBNormal`
- `PointXYZINormal`
- `PointWithRange`
- `PointWithViewpoint`
- `MomentInvariants`
- `PrincipalRadiiRSD`
- `Boundary`
- `PrincipalCurvatures`
- `BounPFHSignature125dary`
- `FPFHSignature33`
- `VFHSignature308`
- `Narf36`
- `BorderDescription`
- `IntensityGradient`
- `Histogram`

Point Cloud Library

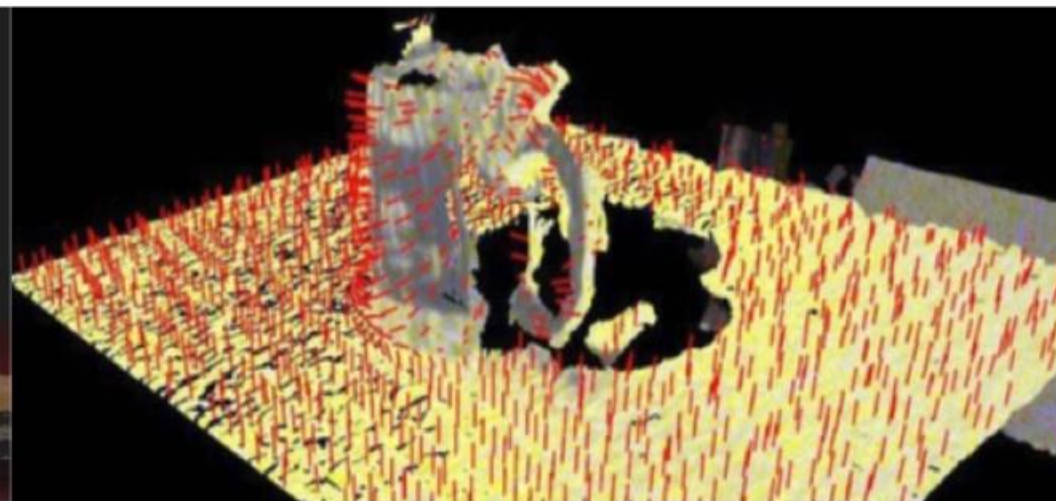
➤ PCL点云数据结构



XYZ
PointXYZ



XYZ+RGB
PointXYZRGBA
PointXYZRGB



XYZ+Normals
PointNormal

Point Cloud Library

➤ PCL点云数据结构-ROS

The ROS PointCloud(2) data format

`sensor_msgs/PointCloud2.msg`

`#This message holds a collection of nD points, as a binary blob.`

`Header header`

`#2D structure of the point cloud. If the cloud is unordered,
#height is 1 and width is the length of the point cloud.`

`uint32 height`

`uint32 width`

`#Describes the channels and their layout in the binary data blob`

`PointField[] fields`

`bool is_bigendian       #Is this data bigendian?`

`uint32 point_step       #Length of a point in bytes`

`uint32 row_step         #Length of a row in bytes`

`uint8[] data            #Actual point data, size is(row_step*height)`

`bool is_dense           #True if there are no invalid points`

Point Cloud Library

➤ PCL点云数据结构-ROS

The ROS PointCloud(2) data format

`sensor_msgs/PointCloud2.msg`

```
#This message holds the description of one point entry in the
#PointCloud2 message format.
```

```
uint8 INT8      = 1
uint8 UINT8     = 2
uint8 INT16     = 3
uint8 UINT16    = 4
uint8 INT32     = 5
uint8 UINT32    = 6
uint8 FLOAT32   = 7
uint8 FLOAT64   = 8

string name      #Name of field
uint32 offset    #Offset from start of point struct
uint8 datatype   #Datatype enumeration see above
uint32 count     #How many elements in field
```

PointField examples:

```
"x",           0,  7,  1
"y",           4,  7,  1
"z",           8,  7,  1
"rgba",       12,  6,  1
"normal_x",   16,  8,  1
"normal_y",   20,  8,  1
"normal_z",   24,  8,  1
"fpfh",       32,  7, 33
```


Point Cloud Library

➤ PCL点云数据存储

The ROS PointCloud(2) bag files

`sensor_msgs/PointCloud2.msg`

```
$ rostopic find sensor_msgs/PointCloud2 | xargs rosrecord -F
foo

[ INFO] [1271297447.656414502]: Recording to foo.bag.
^C
[ INFO] [1271297450.723504983]: Closing foo.bag.
$ rosbag -c foo.bag
bag: foo.bag
version: 1.2
start_time: 1271297447974280542
end_time: 1271297449983577462
length: 2009296920
topics:
- name: /narrow_stereo_textured/points2
  count: 3
  datatype: sensor_msgs/PointCloud2
  md5sum: 1158d486dd51d683ce2f1be655c3c181
```

Point Cloud Library

➤ PCL点云数据存储

点云文件 PCD (Point Cloud Data)

PCD可视化

```
$ pcl_viewer <path-to-pcd-file>
```



```
# Point Cloud Data (PCD) file format v.7
```

```
VERSION .7
```

```
FIELDS x y z rgb
```

```
SIZE 4 4 4 4
```

```
TYPE F F F F
```

```
COUNT 1 1 1 1
```

```
WIDTH 213
```

```
HEIGHT 1
```

```
VIEWPOINT 0 0 0 1 0 0 0
```

```
POINTS 213
```

```
DATA ascii
```

```
0.93773 0.33763 0 4.2108e+06
```

```
0.90805 0.35641 0 4.2108e+06
```

```
0.81915 0.32 0 4.2108e+06
```

```
0.89264 0.21174 0 4.2108e+06
```

```
...
```

```
Or
```

```
...
```

```
DATA binary
```

```
...
```


Point Cloud Library

➤ PCL点云数据存储

点云文件 PCD (Point Cloud Data) Write_pcd.cpp

```
#include<pcl/io/pcdio.h>
#include<pcl/pointtypes.h>
int main (int argc, char **argv){
    pcl::Point Cloud<pcl::PointXYZ> cloud;
    //Fill in the cloud data
    cloud.width= 50;
    cloud.height  = 1 ;
    cloud.isdense = false;
    cloud.points.resize(cloud.width * cloud.height);
    for(size_t i = 0; i<cloud.points.size(); ++i){
        cloud.points[i].x=1024 * rand() / (RAND_MAX + 1.0f);
        cloud.points[i].y=1024 * rand() / (RAND_MAX + 1.0f);
        cloud.points[i].z=1024 * rand() / (RAND_MAX + 1.0f);}
    pcl::io::save_PCDFileASCII("test_pcd.pcd",cloud);
    return(0); }
```

Point Cloud Library

➤ PCL点云数据存储

点云文件 PCD (Point Cloud Data) Read_pcd.cpp

```
#include<pcl/io/pcdio.h>
#include<pcl/pointtypes.h>
int main (int argc, char **argv)
{
    pcl::Point Cloud<pcl::PointXYZ>:: Ptr cloud (new pcl::Point Cloud<pcl::PointXYZ>);
    // Load the file
    if(pcl::io::load PCDFile<pcl::PointXYZ> ("testpcd.pcd", *cloud) == -1)
    {
        PCL_ERROR ( "Couldn't read file test pcd.pcd \n");
        return (-1);
    }
    // Do some processing on the cloud here
    return(0);
}
```

Point Cloud Library

➤ PCL

PCL特性:

- 完全**模板化**的现代c++库，用于3D点云处理
- 使用**SSE**优化(Eigen后端)在现代cpu上进行快速计算
- 使用**OpenMP**和**Intel TBB**进行并行化
- 使用**Boost共享指针**在模块(如算法)之间传递数据
- 开源项目（BSD license），等同OpenCV在图像处理地位



Point Cloud Library

➤ PCL

PCL模板库:

- **libpcl_features:**

许多3D特征 (例如,法向量与曲率(normals and curvatures), 边界点(boundary points), 不变矩(moment invariants), 主曲率(principal curvatures), 点特征直方图Point Feature Histograms (PFH), Fast PFH, ...)

- **libpcl_surface:**

表面重建技术(例如,面元(meshing), 凸包(convex hulls), 移动最小二乘(Moving Least Squares), ...)

- **libpcl_filters:**

点云数据滤波(例如, 下采样(downsampling), 外点剔除(outlier removal), 索引提取(indices extraction), 投影(projections), ...)

- **libpcl_io:**

I/O 操作 (例如, 读写PCD (Point Cloud Data) and BAG files)

- **libpcl_segmentation:**

分割操作(例如,聚类提取(cluster extraction), 采样一致性模型拟合(Sample Consensus model fitting), 多边形棱柱提取(polygonal prism extraction), ...)

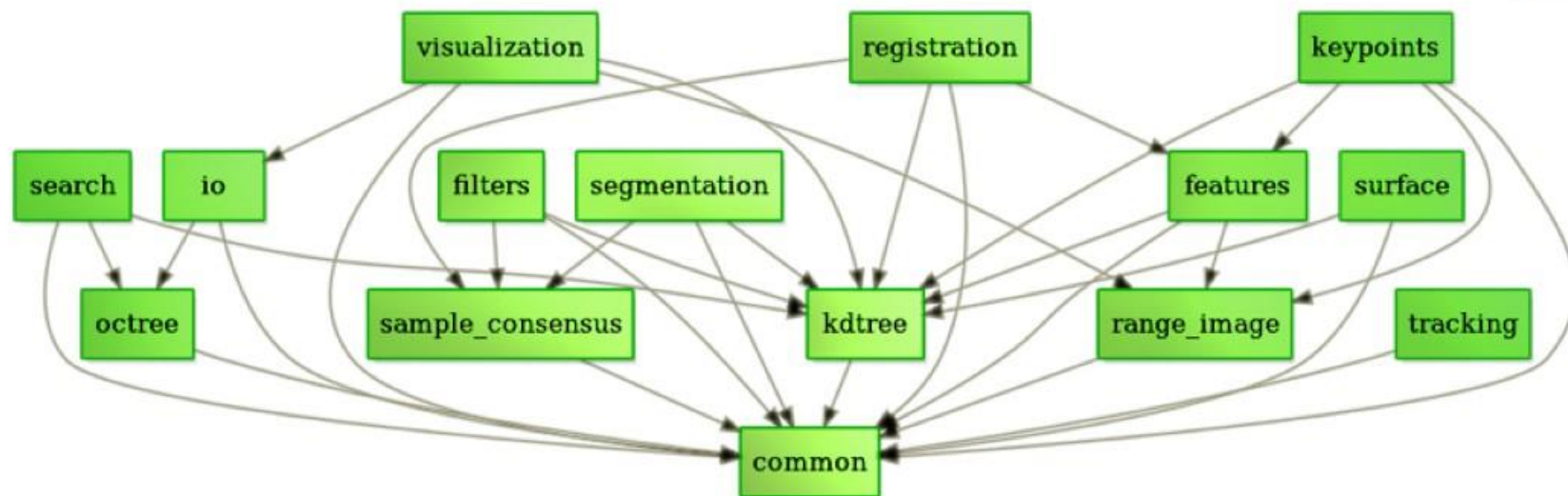
- **libpcl_registration:**

点云注册配准方法(例如,迭代最近点Iterative Closest Point (ICP),正态分布变换Normal Distributions Transform(NDT) 非线性优化(non linear optimizations), ...)

Point Cloud Library

➤ PCL

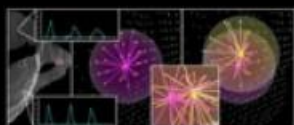
PCL模板库:



filters



features



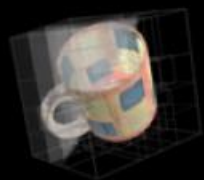
keypoints



registration



kdtree



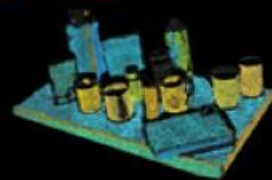
octree



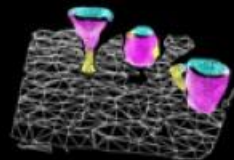
segmentation



sample_consensus



surface



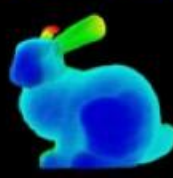
range_image



io



visualization



Point Cloud Library

➤ PCL

PCL Pipeline:

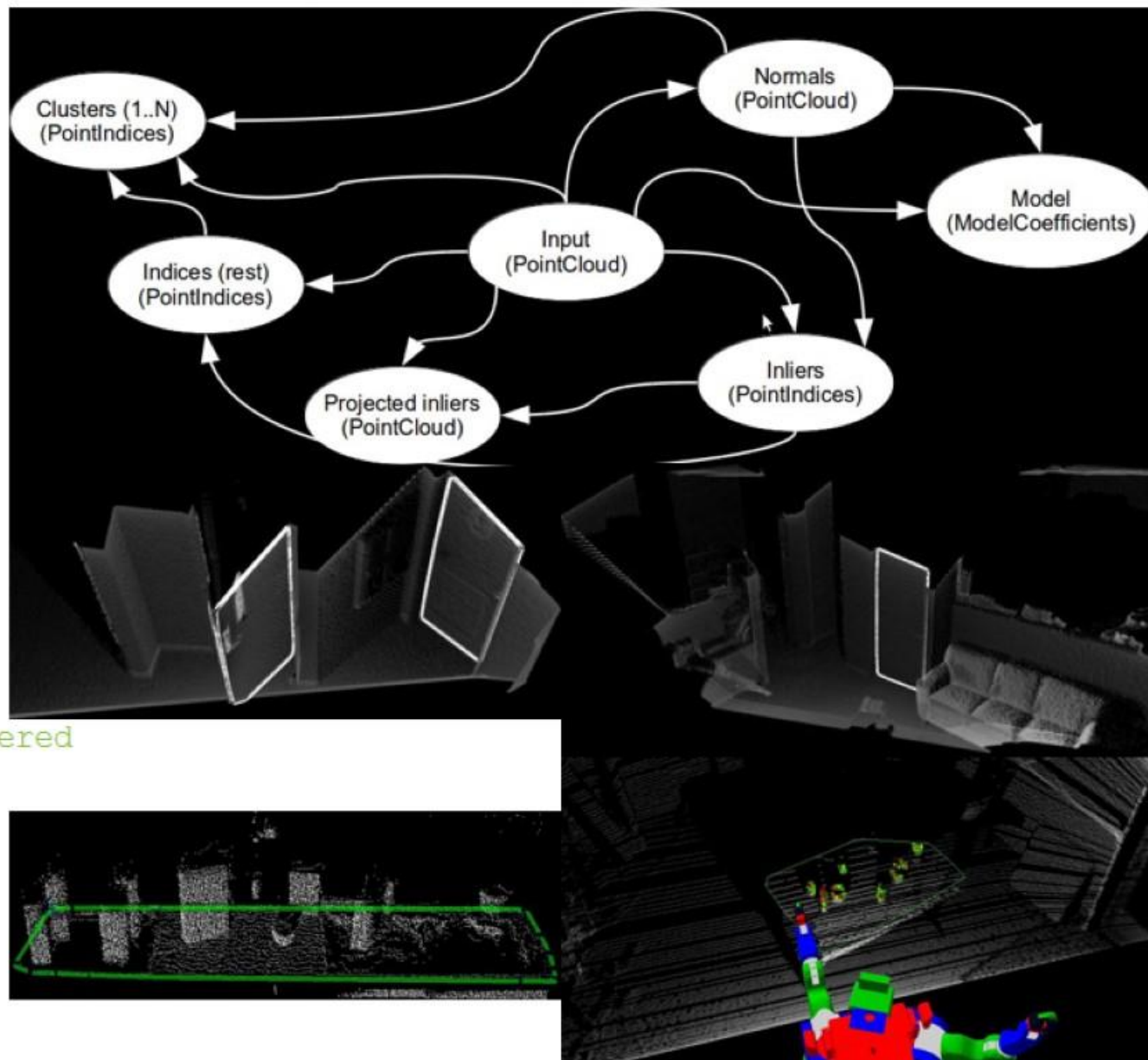
write once, parameterize everywhere

PCL处理管道(pipeline):

- 创建处理对象(例如过滤、特征估计、分割)
- 使用 `setInputCloud` 输入点云数据
- 设置处理对象相关参数(范围、阈值)
- 调用 `compute` (或 `filter`、`segment`) 得到输出

```
pcl::Point Cloud <pcl::PointXYZ>::Ptr cloud
(new pcl::Point Cloud <pcl::PointXYZ>);
pcl::Point Cloud <pcl::PointXYZ>::Ptr cloudfiltered
(new pcl::Point Cloud <pcl::PointXYZ>);
// Pass Through filter
pcl::Pass Through <pcl::PointXYZ> pass;
pass.set InputCloud(cloud);
pass.setFilterFieldName("x");
pass.setFilterLimits(-0.75,0.5);
// pass.setFilterLimitsNegative(true);
pass.filter(*cloudfiltered);
```

PPG: Perception Processing Graphs

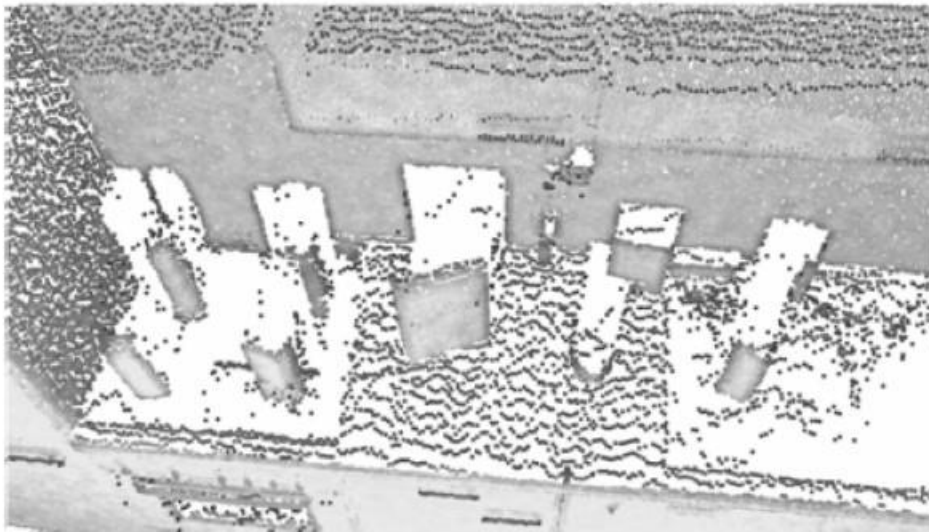
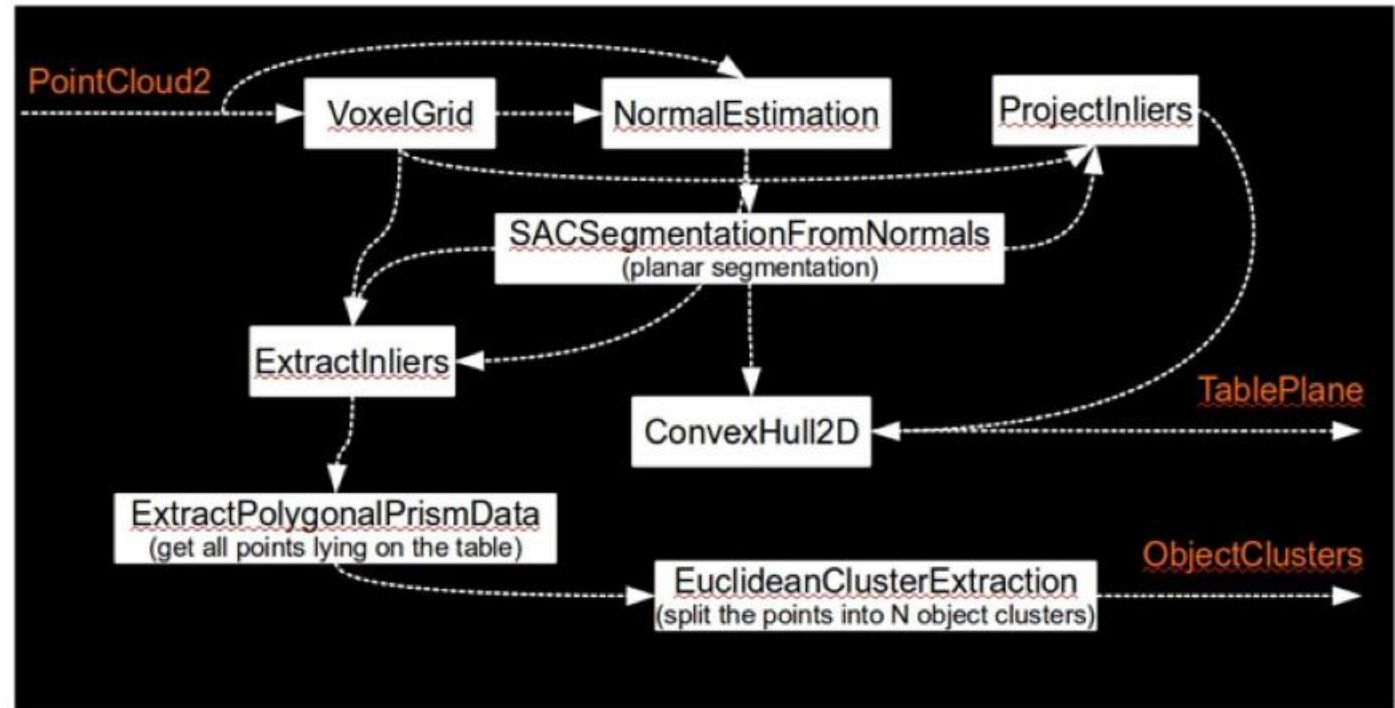


Point Cloud Library

➤ PCL

PCL Pipeline (case):

write once, parameterize everywhere





内容概要

点云滤波方法

点云组织形式与最近邻搜索

点云分割 拟合 聚类方法

激光雷达运动补偿方法

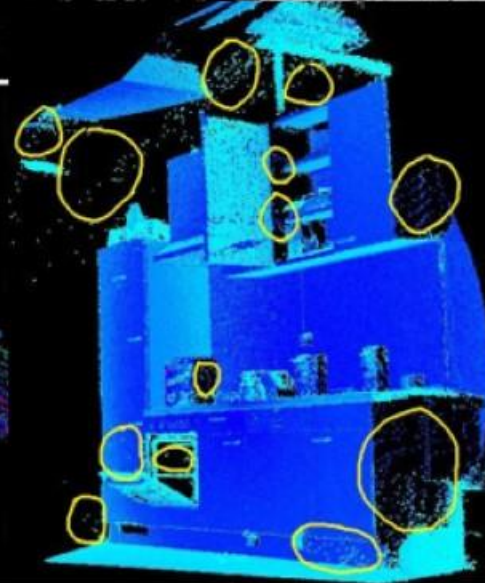
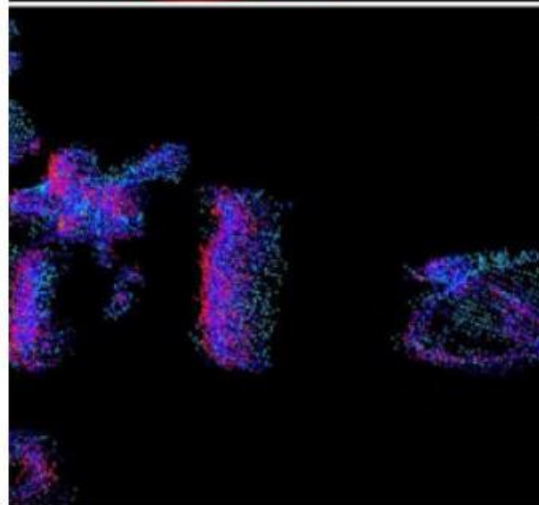
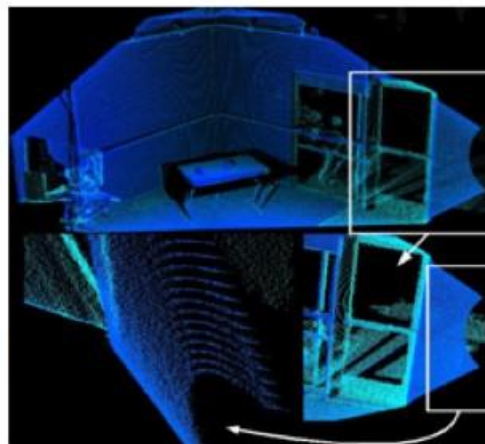
点云去地面原理及方法

点云滤波方法

- 直通滤波器
`pcl::PassThrough<pcl::PointXYZ> pass;`
- 体素滤波器
`pcl::VoxelGrid<pcl::PCLPointCloud2> vox;`
- 统计滤波器
`pcl::StatisticalOutlierRemoval<pcl::PointXYZ> sor;`
- 半径滤波器
`pcl::RadiusOutlierRemoval<pcl::PointXYZ> rador;`
- 均匀采样
`pcl::UniformSampling<pcl::PointXYZ> unisam;`
- 条件滤波器
`pcl::ConditionalRemoval<pcl::PointXYZ> condr;`
- 投影滤波器
`pcl::ProjectInliers<pcl::PointXYZ> proj;`
- 模型滤波器
`pcl::ModelOutlierRemoval<pcl::PointXYZ> modr;`
- 索引提取
`pcl::ExtractIndices<pcl::PointXYZ> extr;`
- 空间裁剪滤波
`pcl::Clipper3D<pcl::PointXYZ>`
`pcl::BoxClipper3D<pcl::PointXYZ>`
`pcl::CropBox<pcl::PointXYZ>`
`pcl::CropHull<pcl::PointXYZ>`
- 双边滤波
`pcl::BilateralFilter<pcl::PointXYZ> bf;`
- 高斯滤波
`pcl::filters::GaussianKernal<PointInT, PointOutT>`

点云数据预处理:

- 点云范围、处理对象区域
- 点云遮挡、局部视角
- 点云数据量大
- 点云采集噪声、平滑处理



点云滤波方法

➤ 直通滤波器

```
pcl::PassThrough<pcl::PointXYZ> pass;
```

```
// 创建滤波器对象
```

```
pcl::PassThrough<pcl::PointXYZ> pass;
```

```
//设置输入点云
```

```
pass.setInputCloud (cloud);
```

```
//滤波字段名被设置为z轴方向
```

```
pass.setFilterFieldName ("z");
```

```
//可接受的范围为 (0.0, 0.5)
```

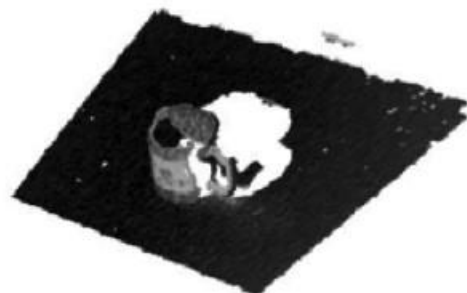
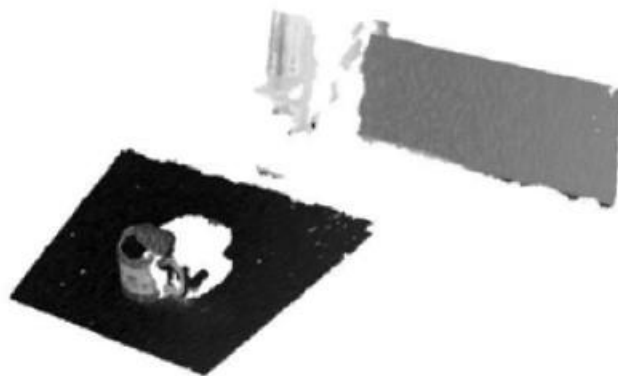
```
pass.setFilterLimits (0.0, 0.5);
```

```
//设置保留范围内 还是 过滤掉范围内
```

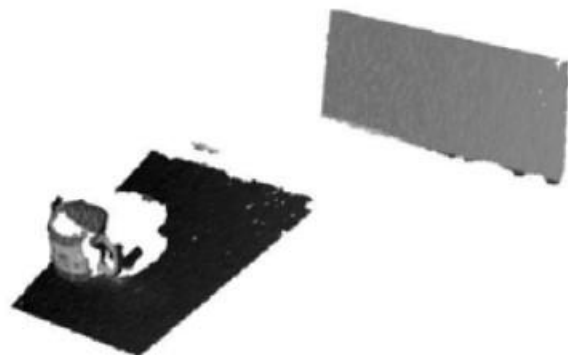
```
//pass.setFilterLimitsNegative (true);
```

```
//执行滤波，保存过滤结果在cloud_filtered
```

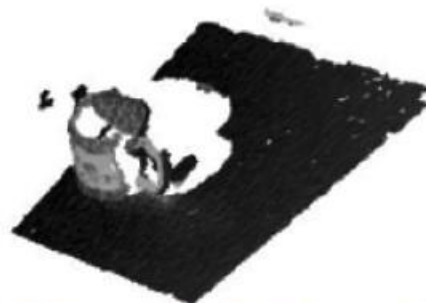
```
pass.filter (*cloud_filtered);
```



```
setFilterFieldName ("z");
```



```
setFilterFieldName ("x");
```



```
setFilterFieldName ("xz");
```

点云滤波方法

➤ 体素滤波器

```
pcl::VoxelGrid<pcl::PCLPointCloud2> vox;
```

// 创建滤波器对象

```
pcl::VoxelGrid<pcl::PointXYZ> vox;
```

//设置输入点云

```
vox.setInputCloud (cloud);
```

//设置滤波时创建的体素体积为1cm的立方体

```
vox.setLeafSize (0.01f, 0.01f, 0.01f);
```

//执行滤波，保存过滤结果在cloud_filtered

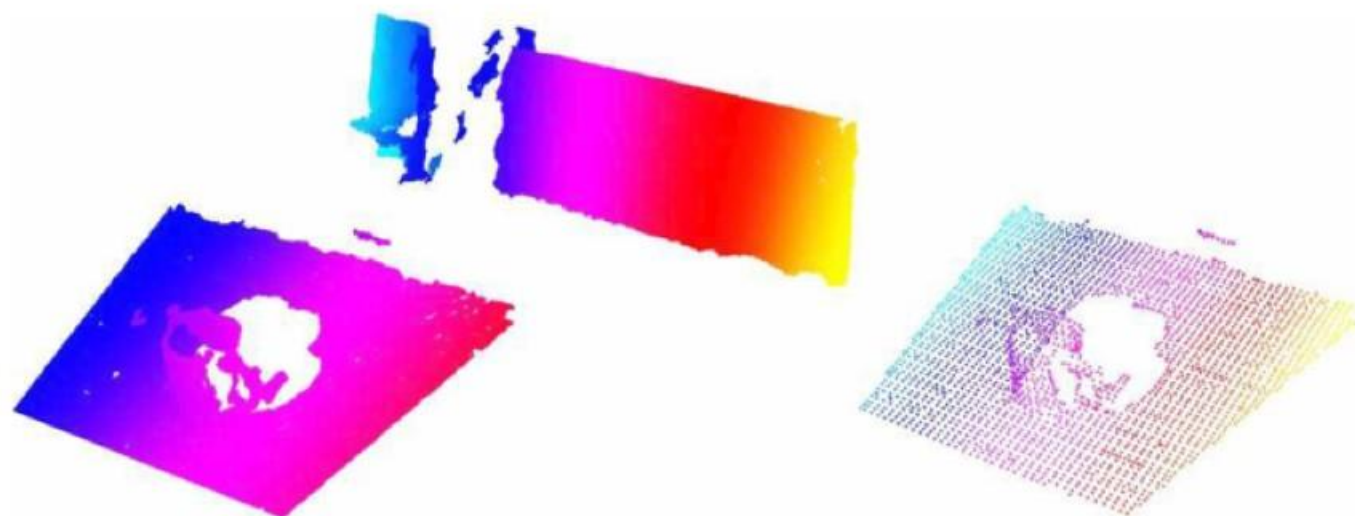
```
vox.filter (*cloud_filtered);
```

栅格重心

```
pcl::ApproximateVoxelGrid
```

1. Compute the min or max of the point set $\{p_1, p_2, \dots, p_N\}$
 $x_{max} = \max(x_1, x_2, \dots, x_N), x_{min} = \min(x_1, x_2, \dots, x_N), y_{max} = \dots$
2. Determine the voxel grid size r
3. Compute the dimension of the voxel grid
 $D_x = (x_{max} - x_{min})/r$
 $D_y = (y_{max} - y_{min})/r$
 $D_z = (z_{max} - z_{min})/r$
4. Compute voxel index for each point
 $h_x = \lfloor (x - x_{min})/r \rfloor$
 $h_y = \lfloor (y - y_{min})/r \rfloor$
 $h_z = \lfloor (z - z_{min})/r \rfloor$
 $h = h_x + h_y * D_x + h_z * D_x * D_y$
5. Sort the points according to the index in Step 4
6. Iterate the sorted points, select points according to Centroid / Random method
0, 0, 0, 0, 3, 3, 3, 8, 8, 8, 8, 8, 8, 8, 8,

<https://www.youtube.com/watch?v=13K62f02000>



点云滤波方法

➤ 统计滤波器

```
pcl::StatisticalOutlierRemoval<pcl::PointXYZ> sor;
```

```
// 创建滤波器对象
```

```
pcl::StatisticalOutlierRemoval<pcl::PointXYZ> sor; if  $d_i > d_{mean} + 1 \times \sigma$ , then remove  $p_i$ , ps.这里的1即通过setStddevMulThresh()设定的1.0
```

```
//设置输入点云
```

```
sor.setInputCloud (cloud);
```

```
//设置在统计时考虑查询点临近点数
```

```
sor.setMeanK (50);
```

```
//设置判断是否为离群点的阈值
```

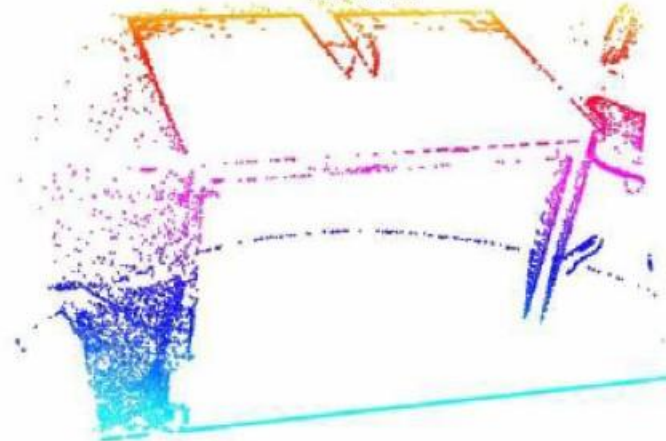
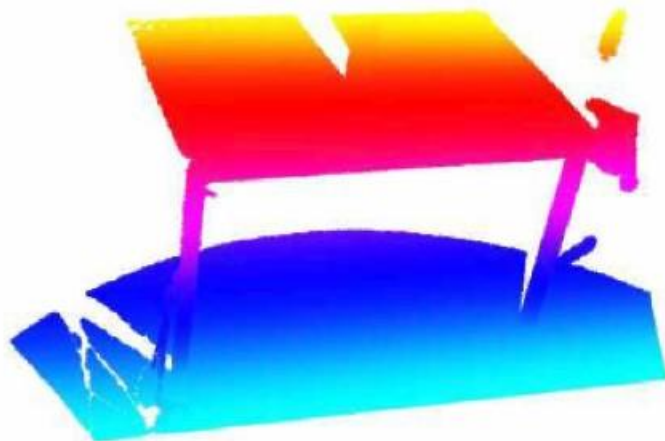
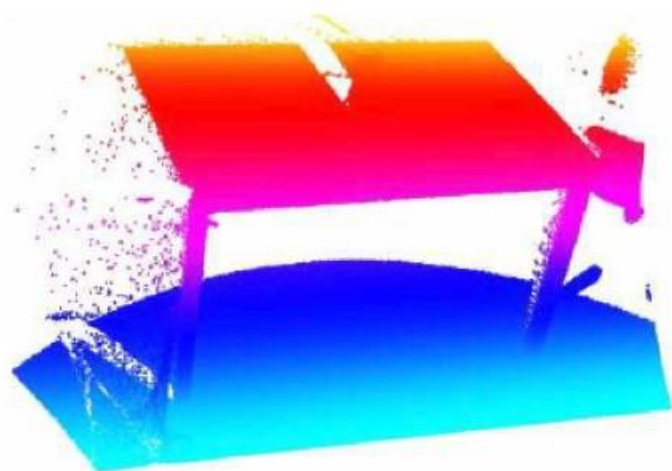
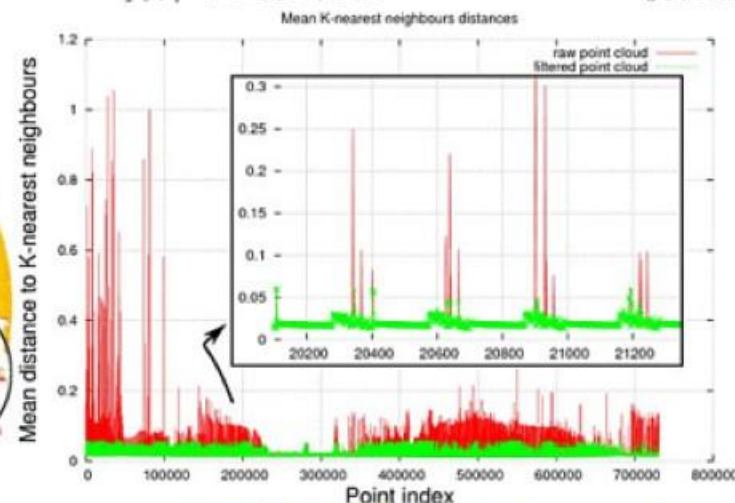
```
sor.setStddevMulThresh (1.0);
```

```
//执行滤波，保存过滤结果在cloud_filtered
```

```
sor.filter (*cloud_filtered);
```

$$d_{mean} = (d_1 + d_2 + d_3 + \dots + d_n) / n \text{ 这里等于50}$$

$$\sigma = \frac{1}{n-1} \sqrt{\sum_{k=1}^n (d_i - d_{mean})^2}$$



点云滤波方法

➤ 半径滤波器

```
pcl::RadiusOutlierRemoval<pcl::PointXYZ> rador;
```

```
//设置半径为0.8的范围内找临近点
```

```
rador.setRadiusSearch(0.8);
```

```
//设置查询点的邻域点集数小于2的删除
```

```
rador.setMinNeighborsInRadius (2);
```

➤ 均匀采样

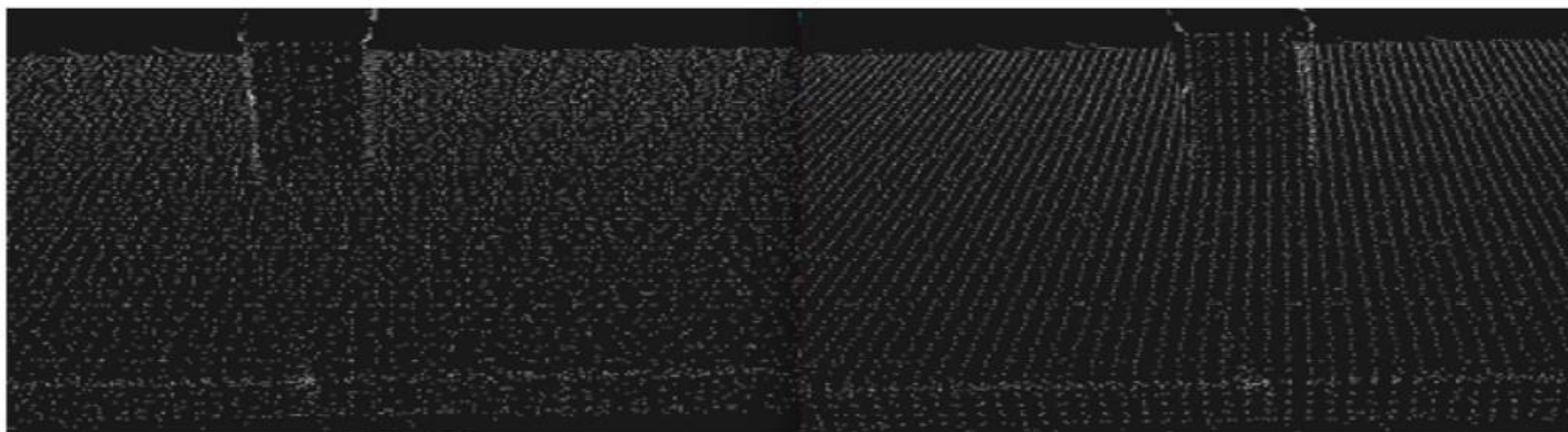
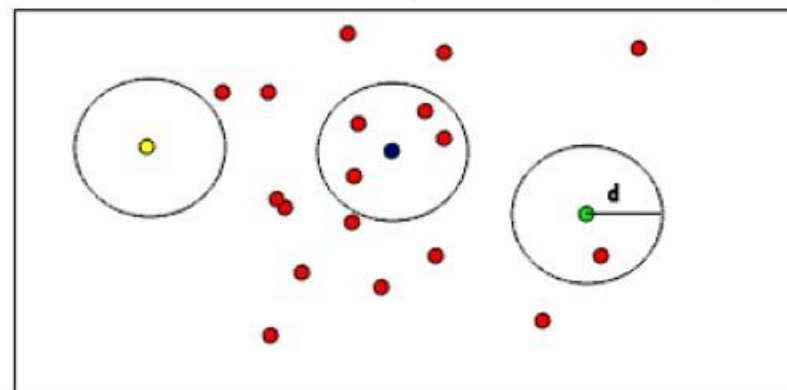
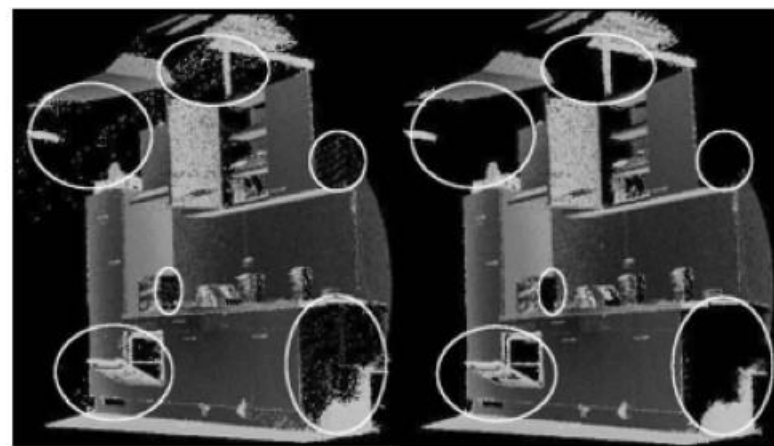
```
pcl::UniformSampling<pcl::PointXYZ> unisam;
```

```
//设置滤波时创建的体素体积为1cm的立方体
```

```
//vox.setLeafSize (0.01f, 0.01f, 0.01f);
```

```
//设置滤波时创建的半径球体
```

```
unisam. setRadiusSearch(0.01f);
```



点云滤波方法

➤ 条件滤波器

```
pcl::ConditionalRemoval<pcl::PointXYZ> cond_r;  
//创建条件定义对象  
pcl::ConditionAnd<pcl::PointXYZ>::Ptr range_cond (new pcl::ConditionAnd<pcl::PointXYZ>());  
//为条件定义对象添加比较算子  
range_cond->addComparison (pcl::FieldComparison<pcl::PointXYZ>::ConstPtr  
(new pcl::FieldComparison<pcl::PointXYZ> ("z", pcl::ComparisonOps::GT, 0.0)));  
//添加在z字段上大于 (pcl::ComparisonOps::GT great Then) 0的比较算子  
//创建条件定义对象 曲率  
//pcl::ConditionOr<PointNormal>::Ptr range_cond (new pcl::ConditionOr<PointNormal> ());  
range_cond->addComparison (pcl::FieldComparison<PointNormal>::ConstPtr  
(new pcl::FieldComparison<PointNormal> ("curvature", pcl::ComparisonOps::GT, threshold)));  
// 创建滤波器并用条件定义对象初始化  
pcl::ConditionalRemoval<pcl::PointXYZ> cond_r;  
//并用条件定义对象初始化  
cond_r.setCondition (range_cond);  
cond_r.setInputCloud (cloud);  
//设置保持点云的结构  
cond_r.setKeepOrganized(true);  
cond_r.filter(*cloud_filtered);
```


点云滤波方法

➤ 索引提取

```
pcl::ExtractIndices<pcl::PointXYZ> extr;
// 创建点云提取对象 设置ExtractIndices的实际参数
pcl::ExtractIndices<pcl::PointXYZ> extr;
// 创建点云索引对象
pcl::PointIndices::Ptr inliers (new pcl::PointIndices());
// 分割点云
// 为了处理点云包含的多个模型，在一个循环中执行该过程并在每次模型被提取后，保存剩余的点进行迭代
//seg.setInputCloud (cloud_filtered); // 设置源 滤波后的点云
//seg.segment (*inliers, *coefficients); // 输入分割系数得到 分割后的点云 索引inliers
// 添加点云索引
inliers->indices->push_back (0);
inliers->indices->push_back (2);
// 提取索引 Extract the inliers
extr.setInputCloud (cloud_filtered);
extr.setIndices (inliers);
extr.setNegative (false);
extr.filter (*cloud_p); // 按索引提取 点云
```

点云滤波方法

- 索引提取

```
pcl::ExtractIndices<pcl::PointXYZ> extr;
```

- 投影滤波器

```
pcl::ProjectInliers<pcl::PointXYZ> proj;
```

- 模型滤波器

```
pcl::ModelOutlierRemoval<pcl::PointXYZ> modr;
```

- 空间裁剪滤波

```
pcl::CropHull<pcl::PointXYZ>
```

```
...
```

```
Demonstration
```

```
...
```




内容概要

点云滤波方法

点云组织形式与最近邻搜索

点云分割 拟合 聚类方法

激光雷达运动补偿方法

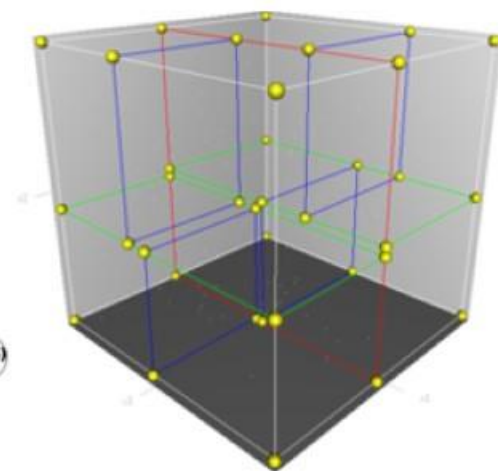
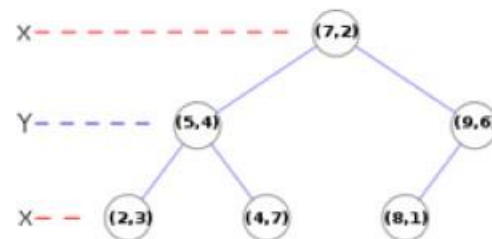
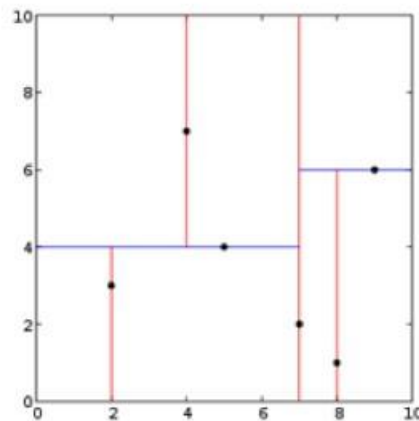
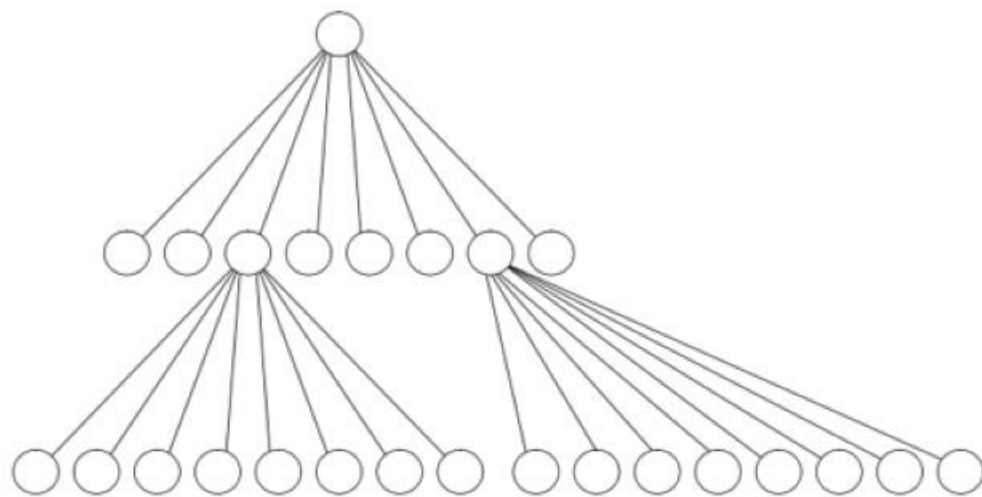
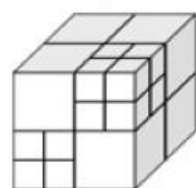
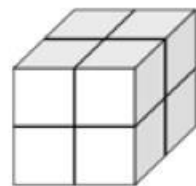
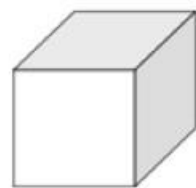
点云去地面原理及方法

点云组织形式与最近邻搜索

- 八叉树(Octree)
- KD树(KdTree)

空间索引结构: 自顶而下逐级划分

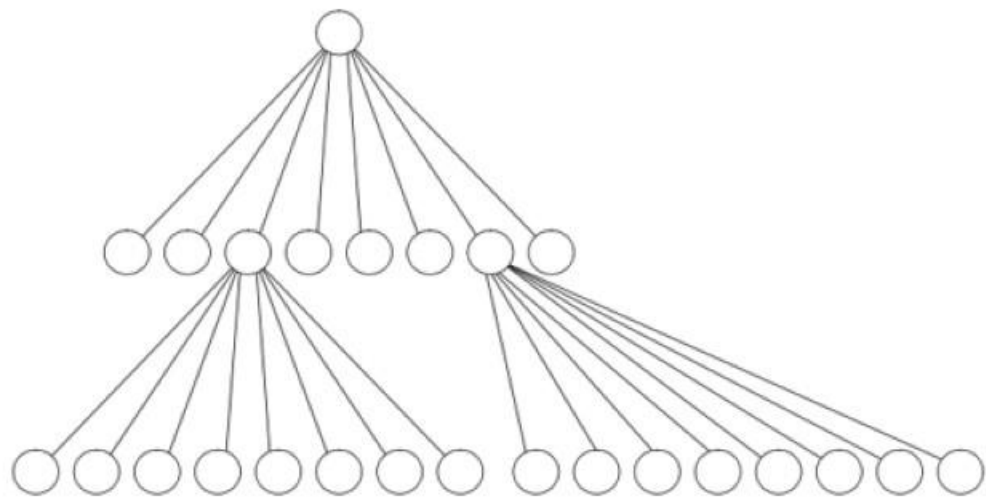
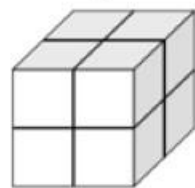
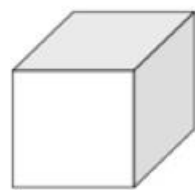
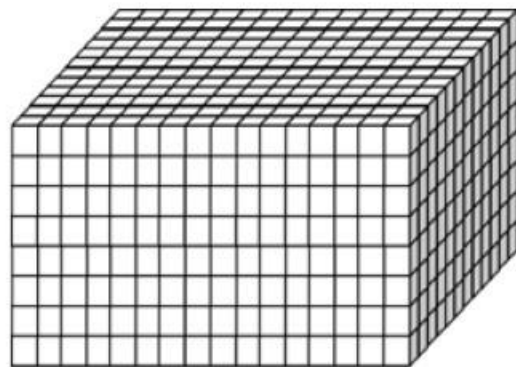
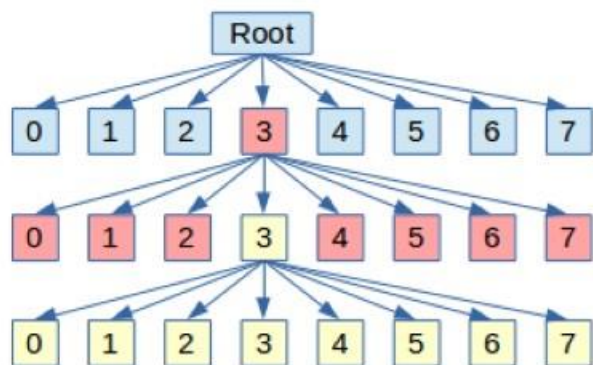
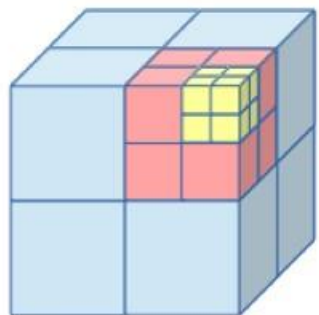
- BSP树
- KD树
- KDB树
- R树
- 四叉树
- 八叉树



点云组织形式与最近邻搜索

➤ 八叉树(Octree)

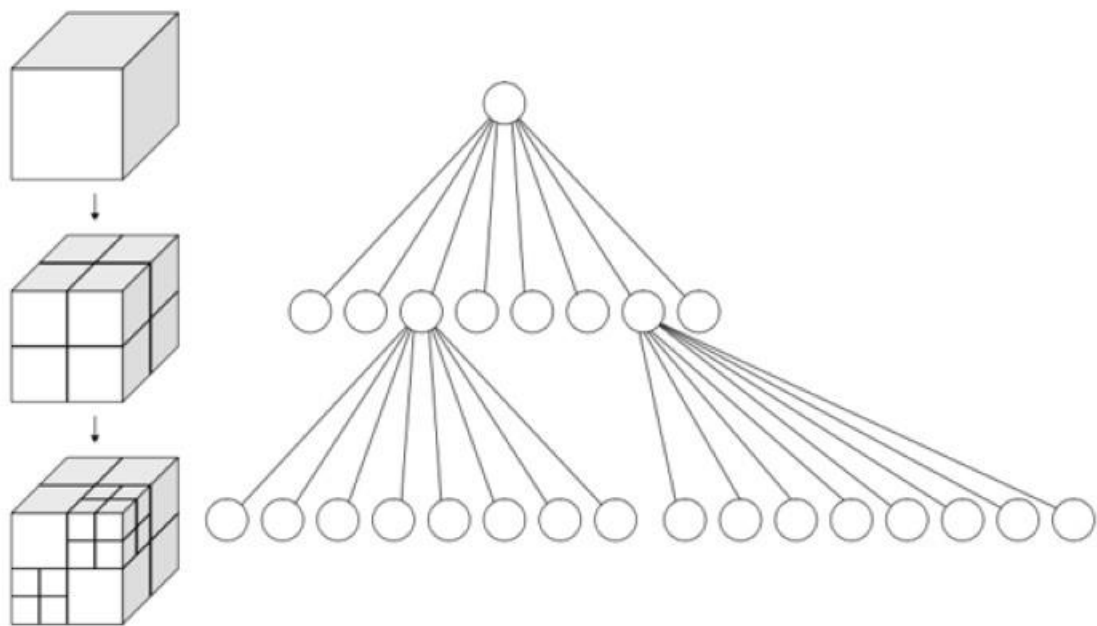
- 体素化——使用空间的均匀分割
- 八叉树——对空间非均匀分割（按需分割）
- 1D数据的2叉树表示
- 2D数据的4叉树表示
- 3D数据的8叉树表示



点云组织形式与最近邻搜索

➤ 八叉树(Octree)

- 构建八叉树

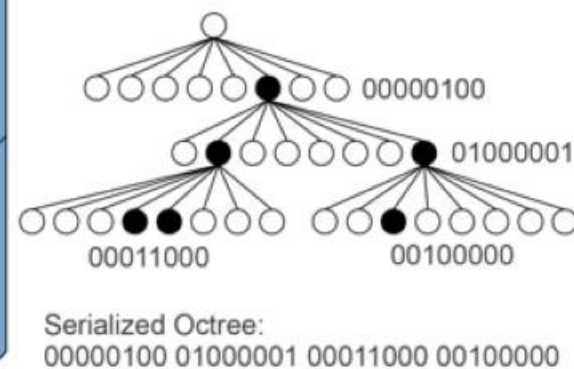
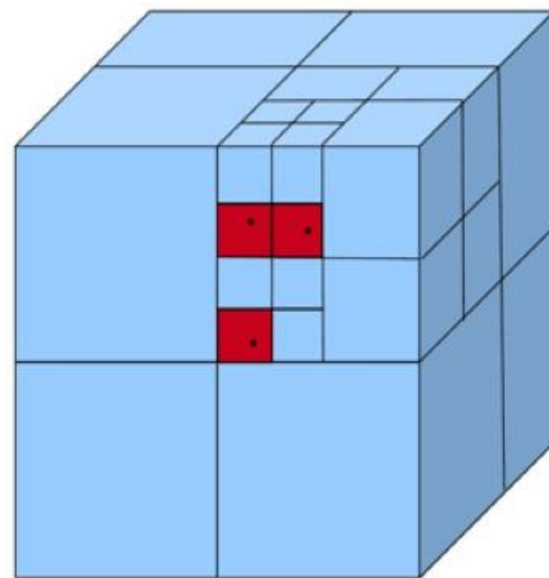
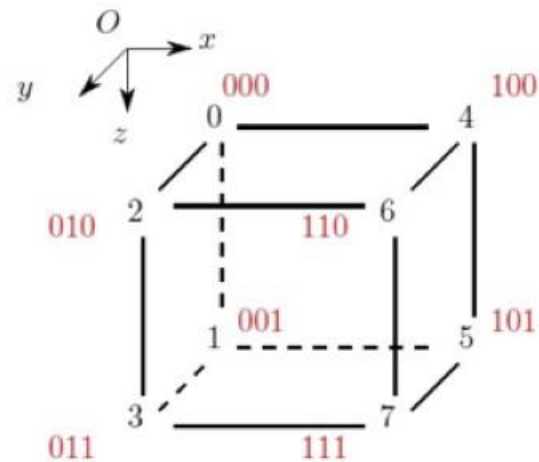
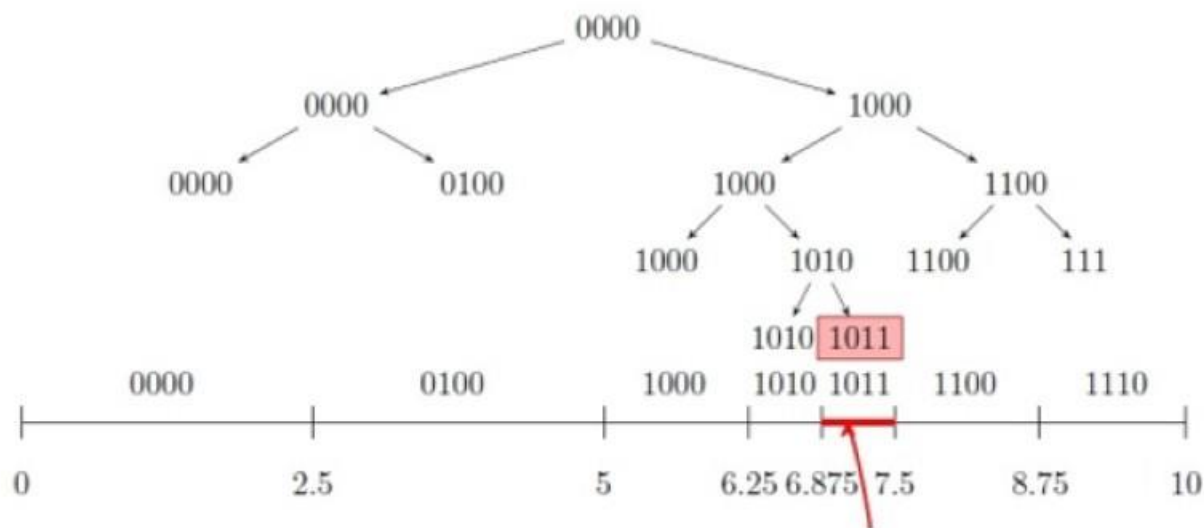


- (1). 设定最大递归深度。
- (2). 找出场景的最大尺寸，并以此尺寸建立第一个立方体。
- (3). 依序将单位元元素丢入能被包含且没有子节点的立方体。
- (4). 若没达到最大递归深度，就进行细分八等份，再将该立方体所装的单位元元素全部分担给八个子立方体。
- (5). 若发现子立方体所分配到的单位元元素数量不为零且跟父立方体是一样的，则该子立方体停止细分，因为跟据空间分割理论，细分的空间所得到的分配必定较少，若是一样数目，则再怎么切数目还是一样，会造成无穷切割的情形。
- (6). 重复3，直达到最大递归深度。

点云组织形式与最近邻搜索

➤ 八叉树(Octree)

- 线性编码构建二叉树
- 线性编码构建八叉树

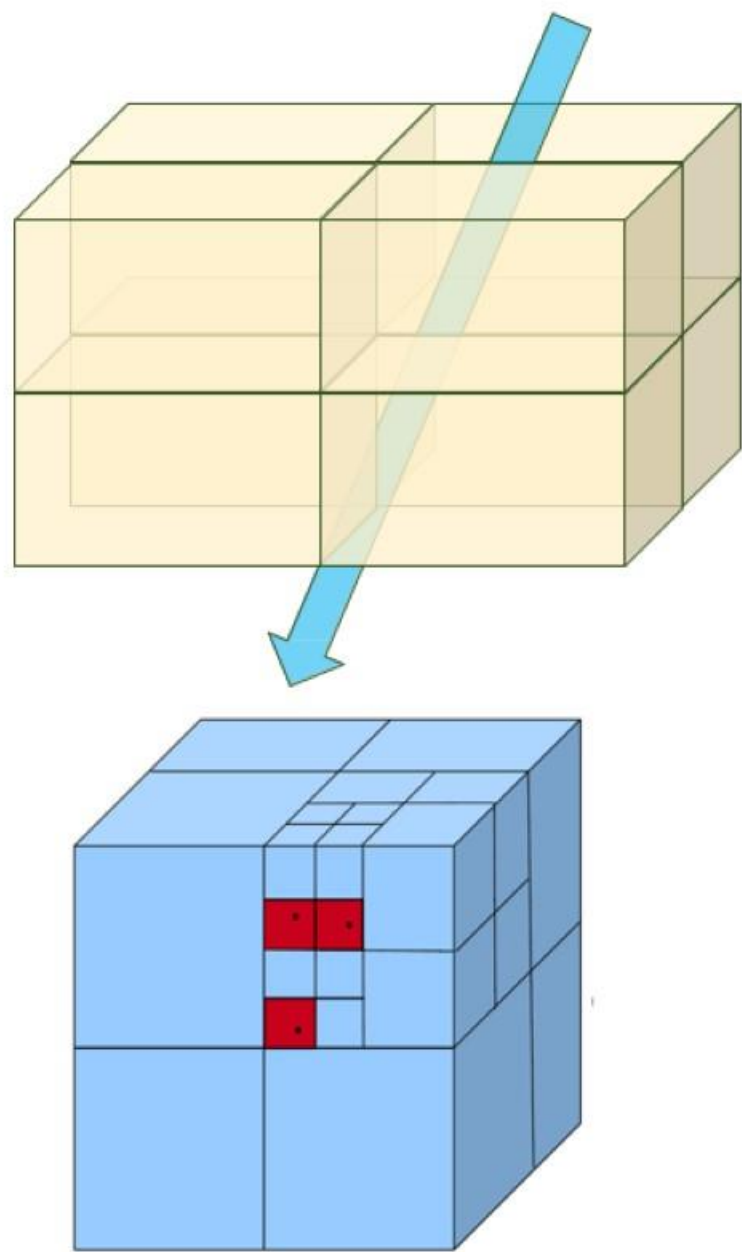


点云组织形式与最近邻搜索

➤ 八叉树(Octree)

点云八叉树应用算法:

- 搜索操作(邻域、半径、体素搜索)
- 降采样(体素网格/体素质心滤波器)
- 点云压缩
- 空间变化检测
- 空间点密度分析
- 占用检查/占位地图
- 碰撞检测
- 点云合并



点云组织形式与最近邻搜索

➤ 八叉树(Octree)

PCL Octree 使用:

- 实例化Octree

```
float voxelSize = 0.01f; //最小体素分辨率  
OctreePointCloud<PointXYZ> octree(voxelSize);
```

- 设置输入点云

```
octree.setInputCloud(cloud);
```

- 定义Octree边界框(可选)

```
//计算输入点云的边界框  
octree.defineBoundingBox();  
//手动定义点云的边界框  
octree.defineBoundingBox(minX, minY, minZ, maxX, maxY, maxZ);
```

- 输入点云添加到Octree

```
octree.addPointsFromInputCloud();
```

- 删除Octree数据结构

```
octree.deleteTree();
```

点云组织形式与最近邻搜索

➤ 八叉树(Octree)

PCL Octree 使用:

- 检查给定点坐标的体素是否存在

```
double X,Y,Z;  bool occupied;  
X = 1.0; Y=2.0; Z=3.0;  
occupied = octree.isVoxelOccupiedAtPoint(X, Y, Z);
```

- 获取所有占用体素的中心点(Voxel grid filter/downsampling)

```
std::vector<PointXYZ> pointGrid;  
octree.getOccupiedVoxelCenters(pointGrid);
```

- 查询点所在的体素内的近邻点

```
std::vector<int> pointIdxVec;  
octree.voxelSearch(searchPoint, pointIdxVec);
```

- 删除体素

```
pcl::PointXYZ point_arg( 1.0, 2.0, 3.0 );  
octree.deleteVoxelAtPoint( point );
```

- 迭代器

```
OctreePointCloud<PointXYZ>::Iterator it (octreeA); //迭代所有octree节点  
OctreePointCloud<PointXYZ>::LeafNodeIterator itL (octreeA); // 迭代所有octree叶节点
```

点云组织形式与最近邻搜索

➤ 八叉树(Octree)

PCL Octree 类型:

- ```
float resolution = 0.01f;
```
- `// 等于OctreePointCloudPointVector<PointXYZ>`  
`OctreePointCloud<PointXYZ> octreeA (resolution);`
  - `// 叶节点存储vector的索引`  
`OctreePointCloudPointVector<PointXYZ> octreeB (resolution);`
  - `// 叶节点存储单个点的索引`  
`OctreePointCloudSinglePoint<PointXYZ> octreeC (resolution);`
  - `//叶节点不存储任何点的信息`  
`OctreePointCloudOccupancy<PointXYZ> octreeD (resolution);`



# 点云组织形式与最近邻搜索

## ➤ 八叉树(Octree)

PCL Octree 邻域搜索:

- 体素内近邻搜索
- 半径内近邻搜索
- K近邻搜索

- 体素内近邻搜索

```
std::vector<int> pointIdxVec;
octree.voxelSearch(searchPoint, pointIdxVec);
```

# 点云组织形式与最近邻搜索

## ➤ 八叉树(Octree)

PCL Octree 邻域搜索:

- 半径内近邻搜索
- 深度优先树搜索
- 在每个节点上查询与搜索半径球体重叠的已占用子体素

### • 半径内近邻搜索

```
std::vector<int> pointIdxRadiusSearch;
```

```
std::vector<float> pointRadiusSquaredDistance;
```

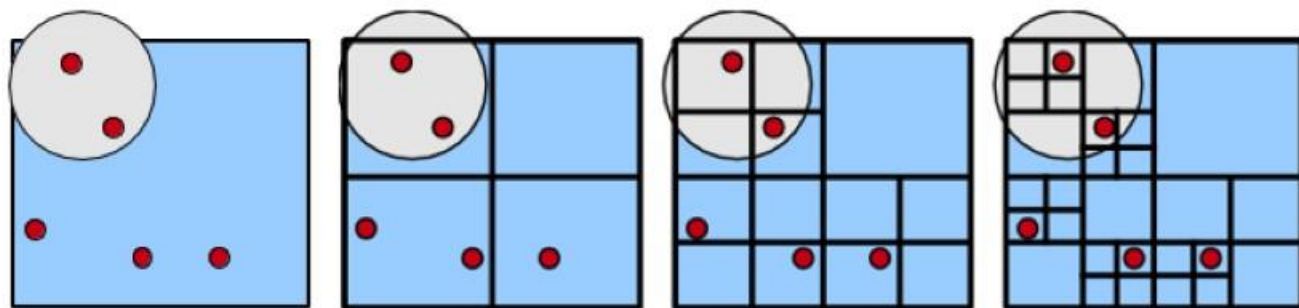
```
float radius = 0.1;
```

- // 扫描所有候选体素

```
if (octree.radiusSearch (searchPoint, radius, pointIdxRadiusSearch,
pointRadiusSquaredDistance) > 0)
{ ... }
```

- // 只扫描“搜索点体素”

- if ( octree.approxNearestSearch (searchPoint, radius, pointIdxRadiusSearch,  
pointRadiusSquaredDistance) > 0 )  
{ ... }



# 点云组织形式与最近邻搜索

## ➤ 八叉树(Octree)

PCL Octree 邻域搜索:

- **K近邻搜索**
- 节点与候选点的优先队列（二叉堆）
- 只查询已占用的体素（最近体素优先）
- 等于半径搜索（半径=到第k个候选点的距离）

- **K近邻搜索**

```
int K = 10;
std::vector<int> pointIdxNKNSearch;
std::vector<float> pointNKNSquaredDistance;
if (octree.nearestKSearch (searchPoint, K, pointIdxNKNSearch,
pointNKNSquaredDistance) > 0)
{
 for (size_t i = 0; i < pointIdxNKNSearch.size(); ++i)
 {
 ...
 }
}
```



# 点云组织形式与最近邻搜索

## ➤ 八叉树(Octree)

PCL Octree 应用:

- 光线跟踪/碰撞检测

```
PointCloud<PointXYZ>::Ptr voxelCenters (new PointCloud<PointXYZ>());
Eigen::Vector3f origin (0.0f, 0.0f, 0.0f);
Eigen::Vector3f direction (0.1f, 0.2f, 0.3f);
if (octree.getIntersectedVoxelCenters(origin,direction,voxelCenters->points)>0)
{ ... }
```

- 密度估计 OctreePointCloudDensity<PointXYZ>

- 空间变化检测

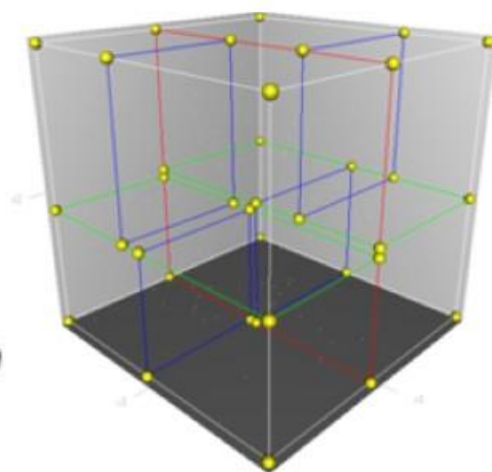
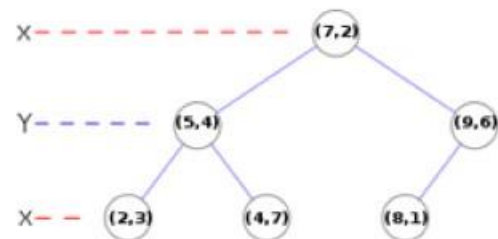
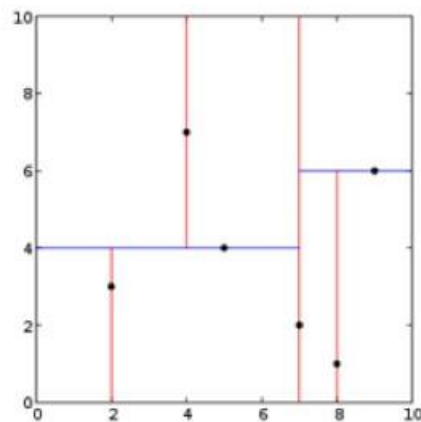
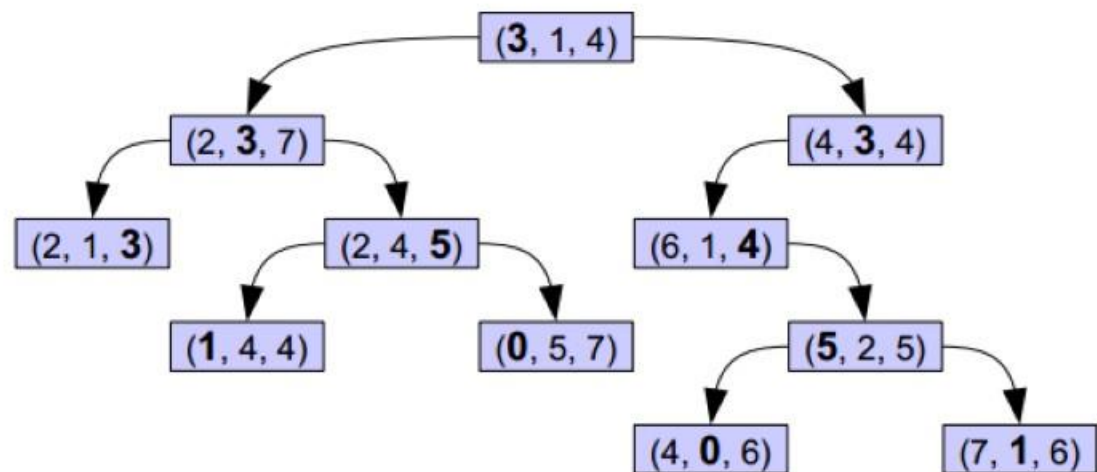
```
octree.setInputCloud (cloudA_ptr); //设置输入点云
octree.addPointsFromInputCloud (); //从输入点云构建八叉树
octree.switchBuffers (); // 交换八叉树缓存, 但是cloudA对应的八叉树结构仍在内存中
octree.setInputCloud (cloudB_ptr); //添加 cloudB到八叉树
octree.addPointsFromInputCloud ();
std::vector<int> newPointIdxVector; //存储新加点的索引的向量
//获取前一cloudA对应的八叉树 在 cloudB对应八叉树中 A内没有的点集
octree.getPointIndicesFromNewVoxels (newPointIdxVector);
```

# 点云组织形式与最近邻搜索

## ➤ KD树(KdTree)

K-Dimensional树:

- 最近邻搜索
- 范围搜索
- 分割k维数据空间



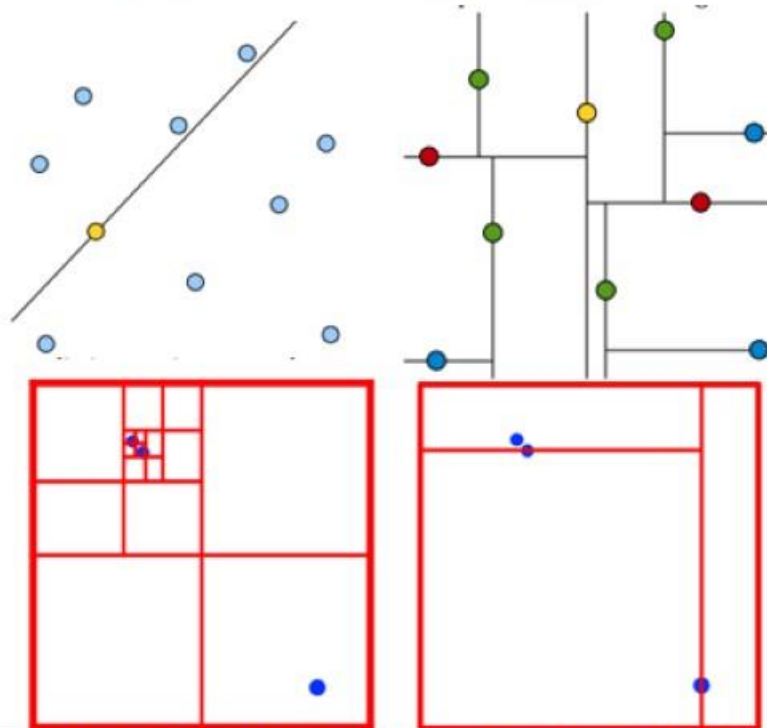
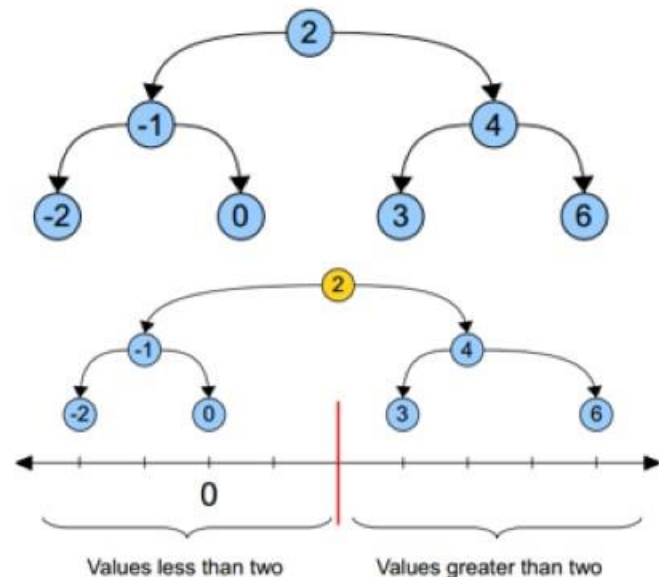
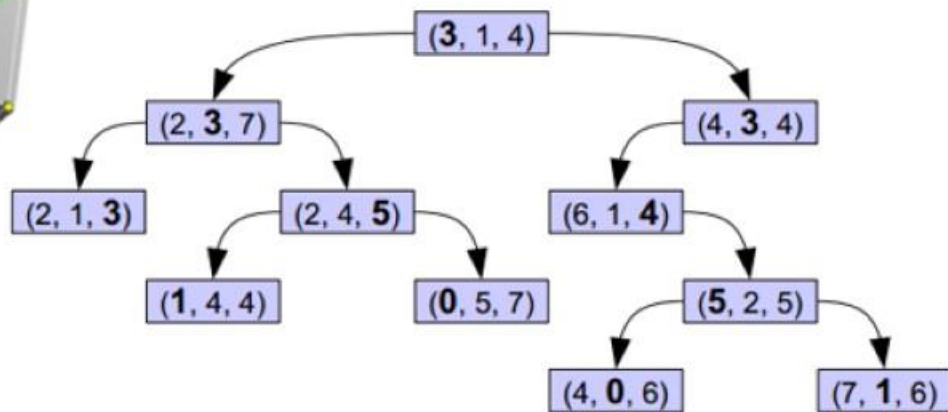
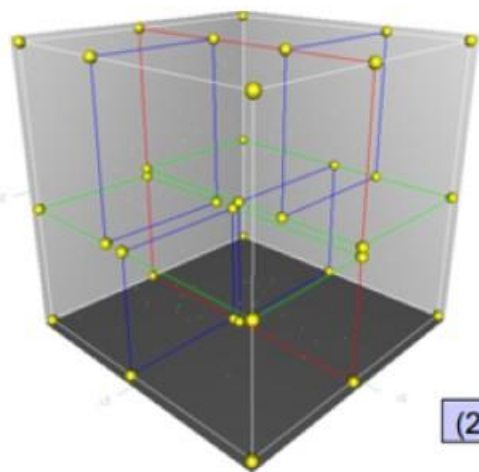
# 点云组织形式与最近邻搜索

## ➤ KD树(KdTree)

K-Dimensional树:

- 概念
- 构建

KDTree就是超平面都垂直于轴的BSPTree





# 点云组织形式与最近邻搜索

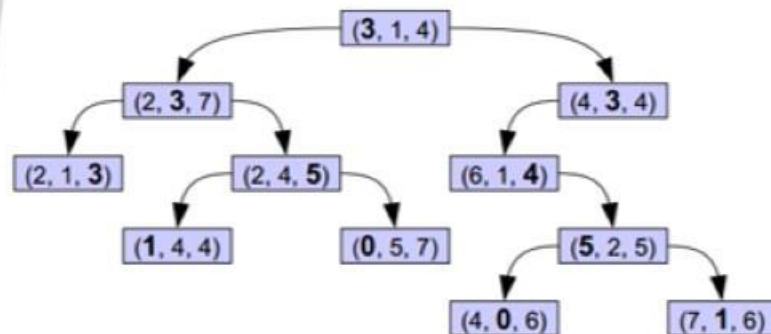
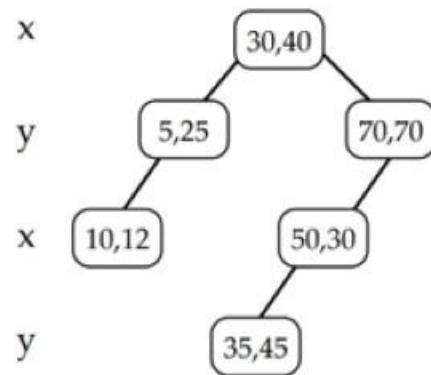
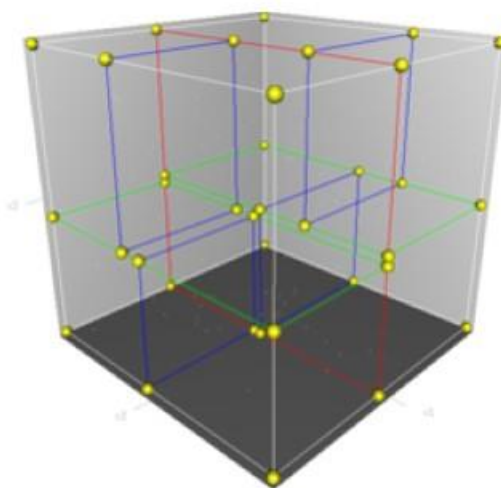
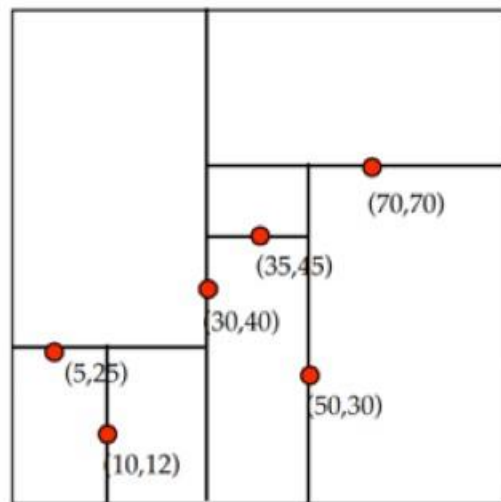
## ➤ KD树(KdTree)

K-Dimensional树:

- 概念
  - 构建
1. 构建包含所有点云的立方体
  2. 对每个包含超多1个点的立方体, 构建分割平面
  3. 两个分割子空间和分割平面上的点构成分支与连接点
  4. 分割子空间, 如果内部点的数量超过1, 继续分割 (即执行2)

分割点选择尽可能均分子空间  
分割平面维度轮转

x,y,z,x,y,z...

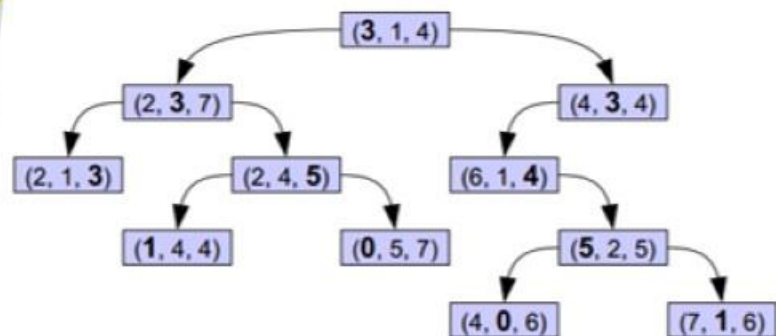
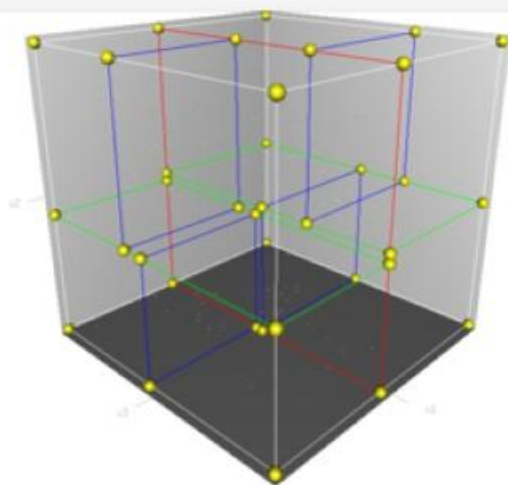


# 点云组织形式与最近邻搜索

## ➤ KD树(KdTree)

K-Dimensional树:

- 概念
- 构建



```
function kdtree (list of points pointList, int depth)
{
 // Select axis based on depth so that axis cycles through all valid values
 var int axis := depth mod k;

 // Sort point list and choose median as pivot element
 select median by axis from pointList;

 // Create node and construct subtree
 node.location := median;
 node.leftChild := kdtree(points in pointList before median, depth+1);
 node.rightChild := kdtree(points in pointList after median, depth+1);
 return node;
}
```

# 点云组织形式与最近邻搜索

## ➤ KD树(KdTree)

K-Dimensional树:

- 搜索最近邻

```
def NN(Point Q, kdTree T, int cd, Rect BB):
```

```
 // if this bounding box is too far, do nothing
 if T == NULL or distance(Q, BB) > best_dist: return
```

```
 // if this point is better than the best:
```

```
 dist = distance(Q, T.data)
```

```
 if dist < best_dist:
```

```
 best = T.data
```

```
 best_dist = dist
```

```
 // visit subtrees in most promising order:
```

```
 if Q[cd] < T.data[cd]:
```

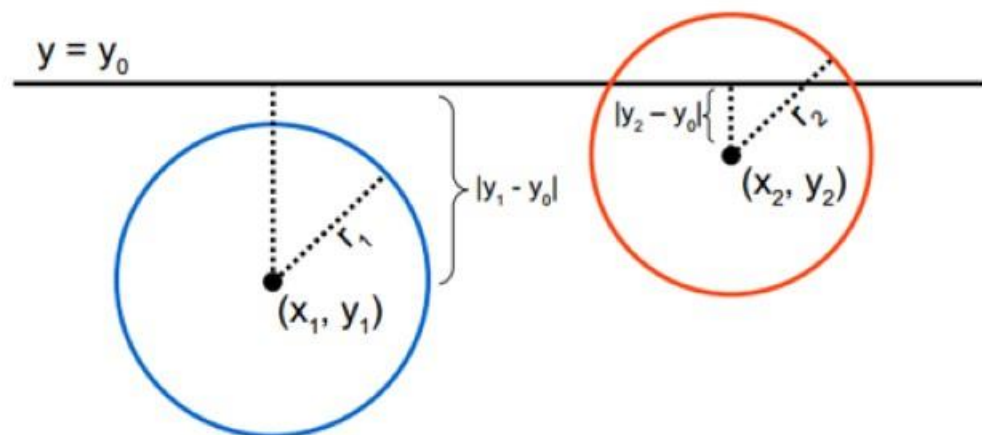
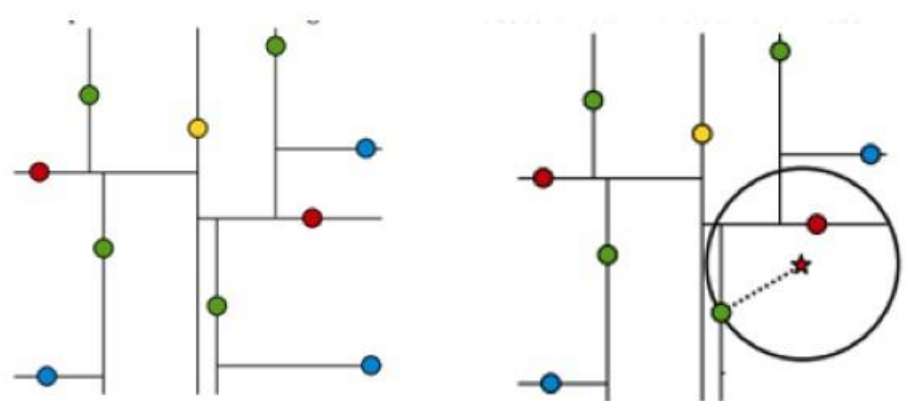
```
 NN(Q, T.left, next_cd, BB.trimLeft(cd, t.data))
```

```
 NN(Q, T.right, next_cd, BB.trimRight(cd, t.data))
```

```
 else:
```

```
 NN(Q, T.right, next_cd, BB.trimRight(cd, t.data))
```

```
 NN(Q, T.left, next_cd, BB.trimLeft(cd, t.data))
```





# 点云组织形式与最近邻搜索

## ➤ KD树(KdTree)

K-Dimensional树:

- 搜索最近邻

查找(2.1,3.1)

(7,2)→(5,4)→(2,3) dist=0.141

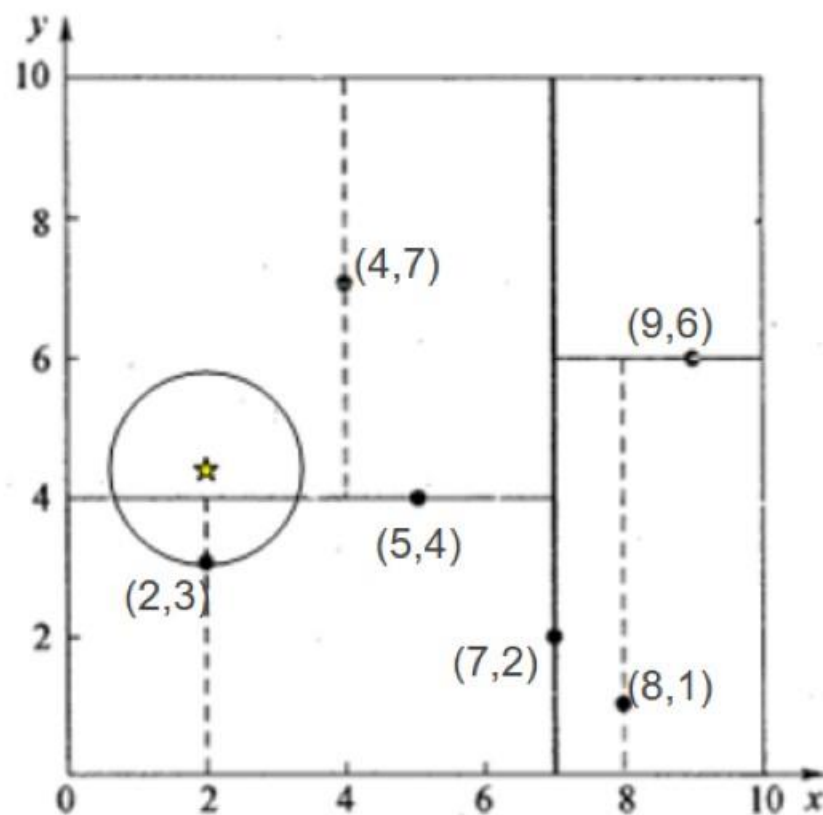
查找(2,4.5)

(7,2)→(5,4)→(4,7) dist=3.04(与(5,4)) 3.202(与(4,7))

div\_dist=0.5

dist=1.5(与(3,2))

- 搜索k近邻  
优先队列



# 点云组织形式与最近邻搜索

## ➤ KD树(KdTree)

PCL KdTree 邻域搜索:

- K近邻搜索
- 半径内近邻搜索
- **K近邻搜索**

```
pcl::KdTreeFLANN<pcl::PointXYZ> kdtree;
kdtree.setInputCloud (cloud);
pcl::PointXYZ searchPoint;
int K = 10;
std::vector<int> pointIdxNKNSearch(K);
std::vector<float> pointNKNSquaredDistance(K);
if (kdtree.nearestKSearch (searchPoint, K, pointIdxNKNSearch,
pointNKNSquaredDistance) > 0)
{
 for (size_t i = 0; i < pointIdxNKNSearch.size(); ++i)
 {
 ...
 }
}
```

# 点云组织形式与最近邻搜索

## ➤ KD树(KdTree)

PCL KdTree 邻域搜索:

- K近邻搜索
- 半径内近邻搜索
- 半径内近邻搜索

```
pcl::KdTreeFLANN<pcl::PointXYZ> kdtree;
kdtree.setInputCloud (cloud);
pcl::PointXYZ searchPoint;
float radius = 256.0f ;
std::vector<int> pointIdxRadiusSearch;
std::vector<float> pointRadiusSquaredDistance;
if (kdtree.radiusSearch (searchPoint, radius, pointIdxRadiusSearch,
pointRadiusSquaredDistance) > 0)
{
 for (size_t i = 0; i < pointIdxRadiusSearch.size(); ++i)
 {
 ...
 }
}
```



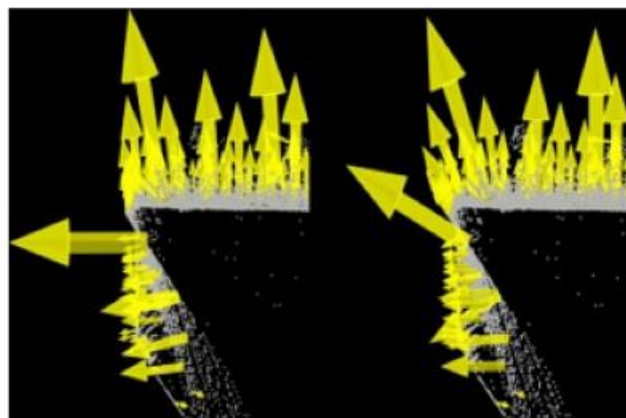
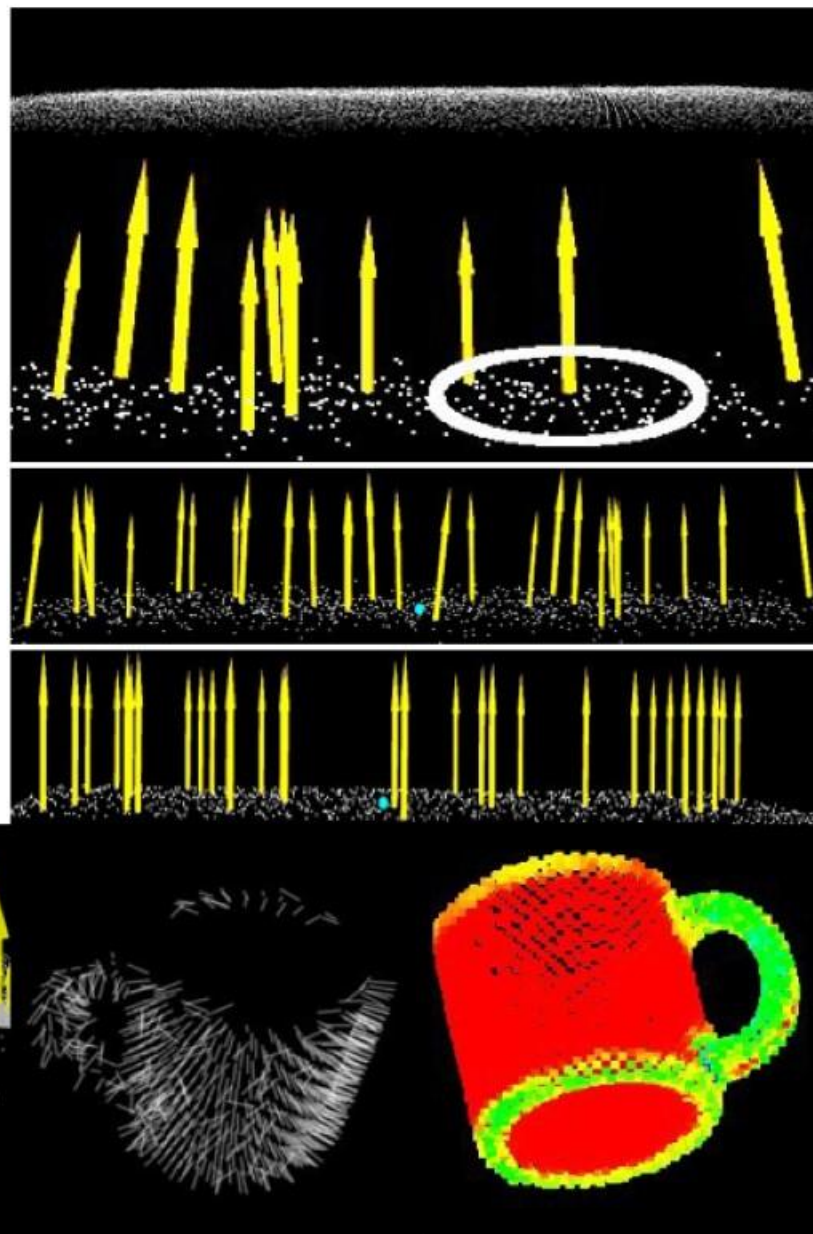
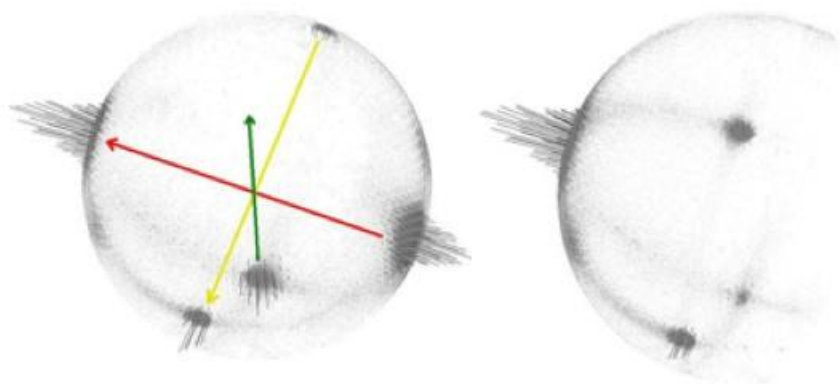
# 点云组织形式与最近邻搜索

## ➤ KD树(KdTree)与法向量估计

PCL KdTree 用于法向量估计 Normal Estimation:

$$C = \frac{1}{k} \sum_{i=1}^k (p_i - \bar{p}) \cdot (p_i - \bar{p})^T, C \cdot \vec{v}_j = \lambda_j \cdot \vec{v}_j, j \in \{0, 1, 2\}$$

$$\sigma = \frac{\lambda_0}{\lambda_0 + \lambda_1 + \lambda_2}$$



# 点云组织形式与最近邻搜索

## ➤ KD树(KdTree)与法向量估计

PCL KdTree 用于法向量估计 Normal Estimation:

- 创建特征估计对象

```
pcl::NormalEstimation<pcl::PointXYZ, pcl::Normal> ne;
```

- 设置用于底层邻居搜索的搜索方法

创建一个k-D树并将其传递给特性估计器

```
pcl::search::KdTree<pcl::PointXYZ>::Ptr tree (new
pcl::search::KdTree<pcl::PointXYZ> ());
ne.setSearchMethod (tree);
```

- 指定计算特征时使用的局部邻域的大小

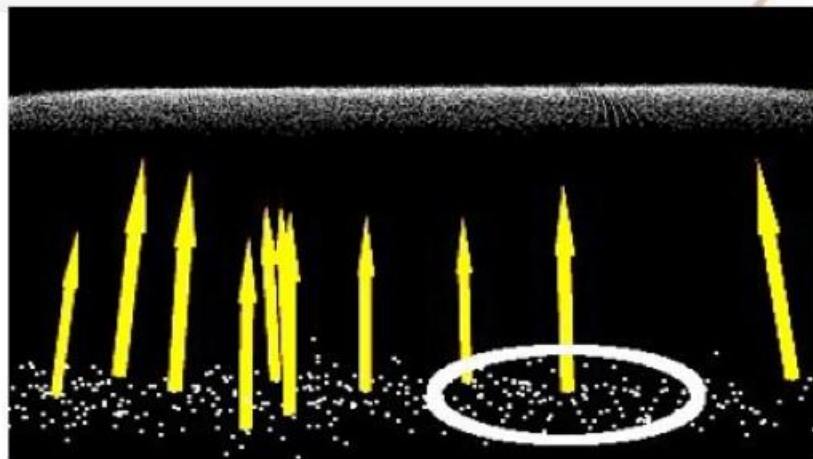
```
ne.setRadiusSearch (r); 或 ne.setNearestKSearch (k)
```

- 设置输入点云

```
ne.setInputCloud (cloud);
```

- 计算

```
pcl::PointCloud<pcl::Normal>::Ptr cloud_normals (new
pcl::PointCloud<pcl::Normal>);
ne.compute (*cloud_normals);
```





# 点云组织形式与最近邻搜索

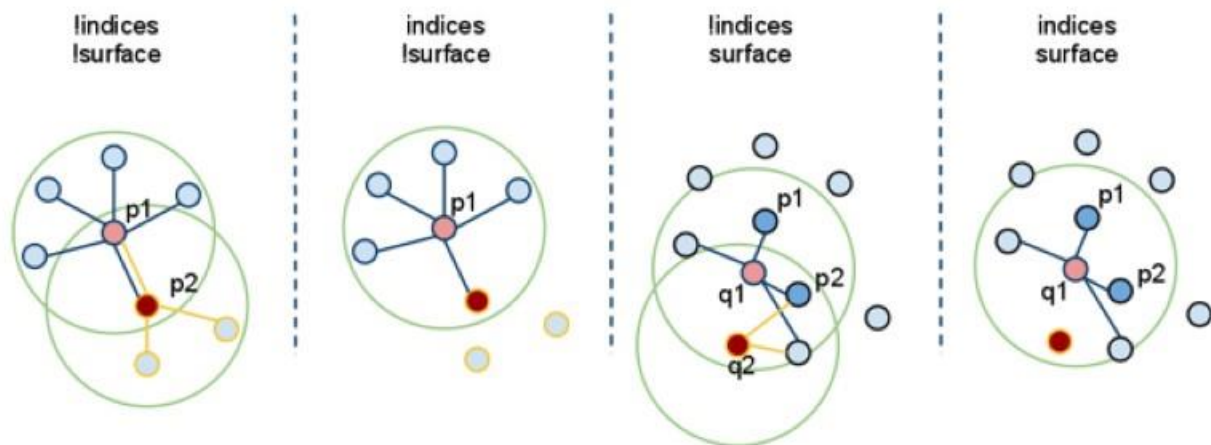
## ➤ KD树(KdTree)与法向量估计

PCL KdTree 用于法向量估计 Normal Estimation:

```
pcl::NormalEstimation<pcl::PointXYZ,
pcl::Normal> ne;
ne.setInputCloud (cloud);
//设置查询setIndices
pcl::shared_ptr<std::vector<int> >
indicesptr (new std::vector<int>
(indices));
ne.setIndices (indicesptr);
//设置KdTree
pcl::search::KdTree<pcl::PointXYZ>::Ptr
tree (new
pcl::search::KdTree<pcl::PointXYZ> ());
ne.setSearchMethod (tree);
//设置半径
ne.setRadiusSearch (0.03);
```

```
<setInputCloud(),
setIndices(),
setSearchSurface>
```

$P=\{p_1, p_2, \dots, p_n\}$  and  $Q=\{q_1, q_2, \dots, q_n\}$



|                          | setIndices = False                                     | setIndices = True                                    |
|--------------------------|--------------------------------------------------------|------------------------------------------------------|
| setSearchSurface = False | compute on <b>all points</b> , using <b>all points</b> | compute on <b>a subset</b> , using <b>all points</b> |
| setSearchSurface = True  | compute on <b>all points</b> , using <b>a subset</b>   | compute on <b>a subset</b> , using <b>a subset</b>   |



# 点云组织形式与最近邻搜索

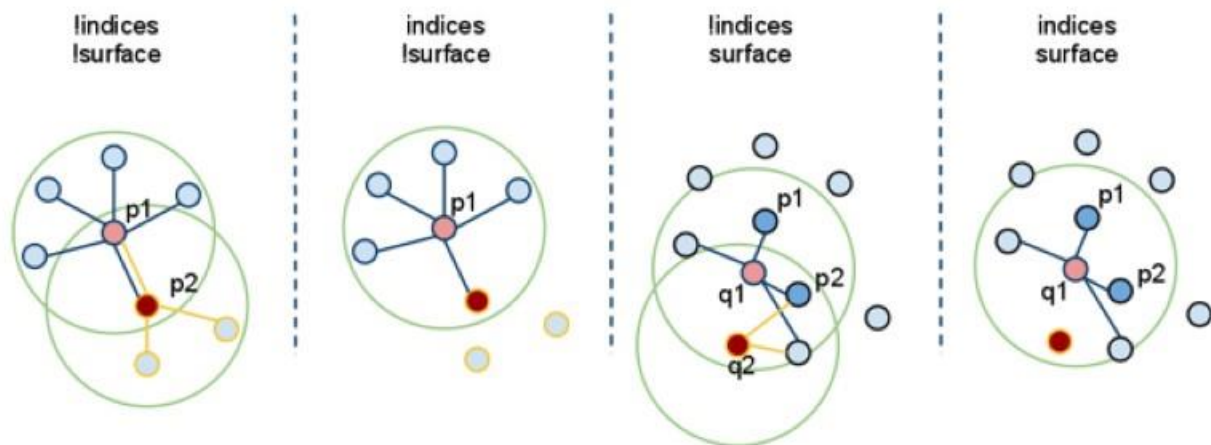
## ➤ KD树(KdTree)与法向量估计

PCL KdTree 用于法向量估计 Normal Estimation:

```
pcl::NormalEstimation<pcl::PointXYZ,
pcl::Normal> ne;
ne.setInputCloud (cloud_downsampled);
//设置查询setSearchSurface
ne.setSearchSurface (cloud);
//设置KdTree
pcl::search::KdTree<pcl::PointXYZ>::Ptr
tree (new
pcl::search::KdTree<pcl::PointXYZ> ());
ne.setSearchMethod (tree);
//设置半径
ne.setRadiusSearch (0.03);
```

```
<setInputCloud(),
setIndices(),
setSearchSurface>
```

$P=\{p_1, p_2, \dots, p_n\}$  and  $Q=\{q_1, q_2, \dots, q_n\}$



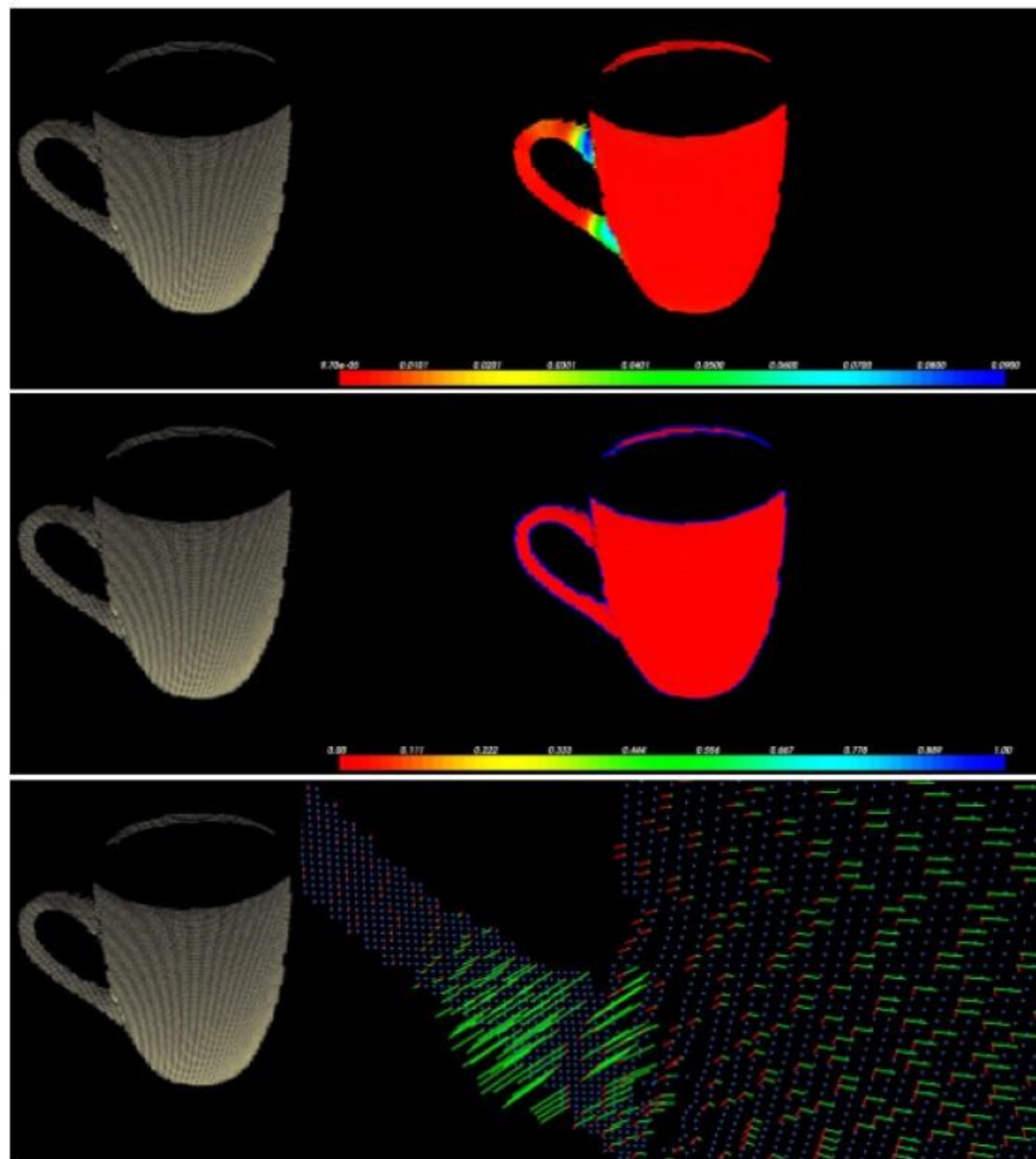
|                          | setIndices = False                                     | setIndices = True                                    |
|--------------------------|--------------------------------------------------------|------------------------------------------------------|
| setSearchSurface = False | compute on <b>all points</b> , using <b>all points</b> | compute on <b>a subset</b> , using <b>all points</b> |
| setSearchSurface = True  | compute on <b>all points</b> , using <b>a subset</b>   | compute on <b>a subset</b> , using <b>a subset</b>   |

# 点云组织形式与最近邻搜索

## ➤ KD树(KdTree)与法向量估计

PCL KdTree 用于法向量估计 Normal Estimation:

- `pcl::NormalEstimation<T> p;`
- `pcl::BoundaryEstimation<T,N> p;`
- `pcl::PrincipalCurvaturesEstimation<T,N> p;`





# 内容概要

点云滤波方法

点云组织形式与最近邻搜索

点云分割 拟合 聚类方法

激光雷达运动补偿方法

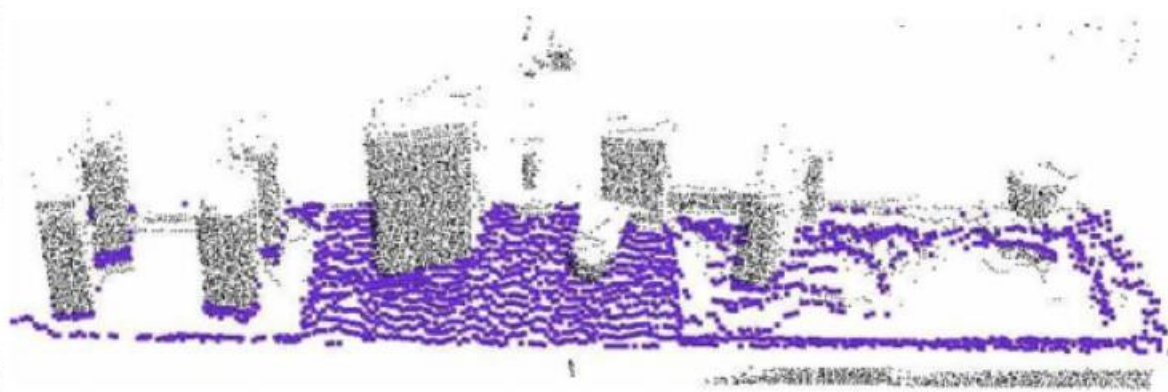
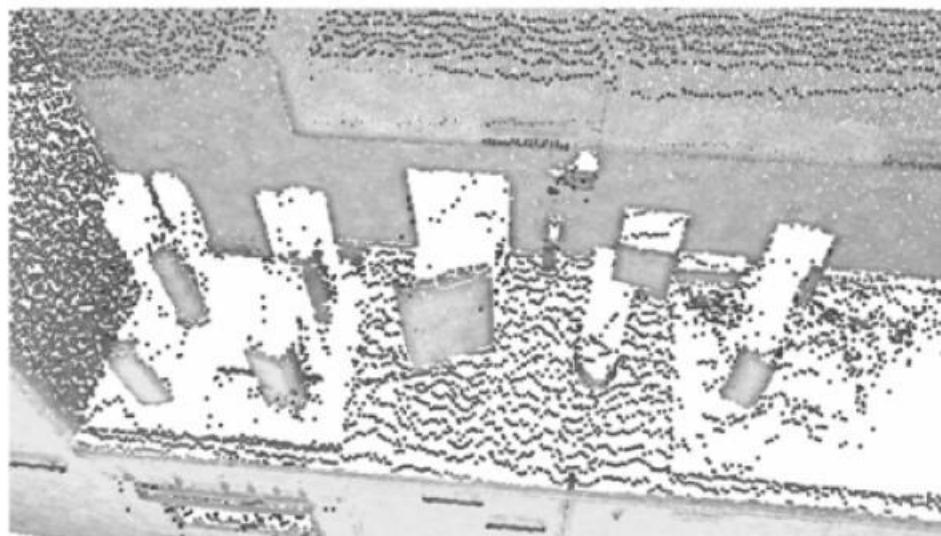
点云去地面原理及方法



# 点云分割 拟合 聚类方法

## ➤ 分割 (segmentation)

- 几何分割
- 语义分割

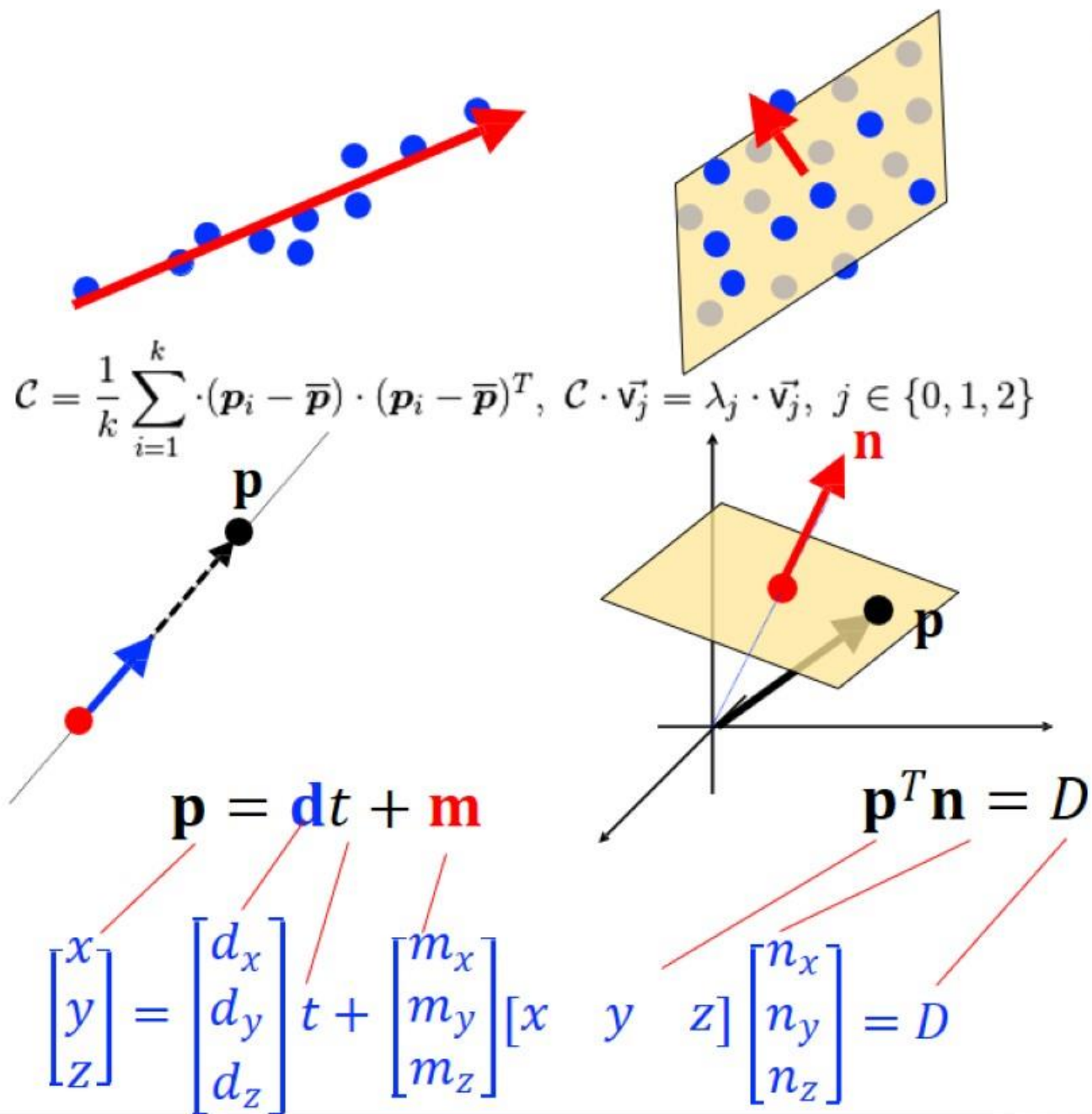


# 点云分割 拟合 聚类方法

## ➤ 分割拟合

分割元素:

- 线
- 平面
- 球体
- 环
- .....



# 点云分割 拟合 聚类方法

## ➤ 分割拟合

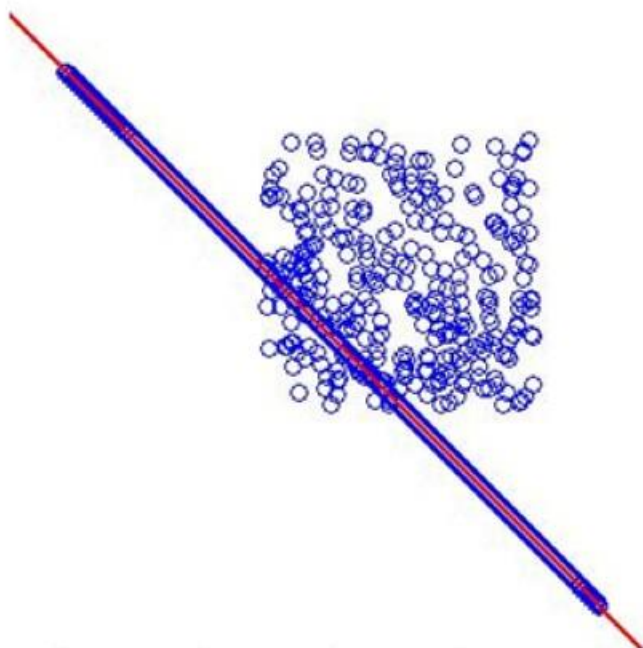
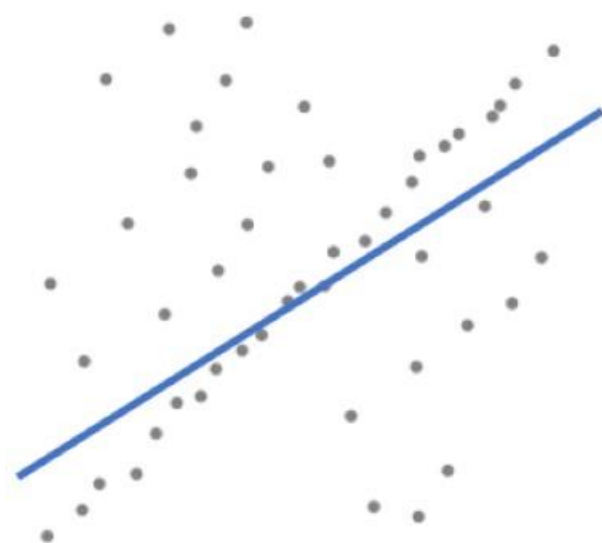
最小二乘法?

随机采样一致性:

RANSAC (Random Sample Consensus)

是一种鲁棒模型拟合的随机算法:

- |                   |            |
|-------------------|------------|
|                   | 线模型        |
| • 选择样本集           | 2个点        |
| • 计算模型            | 线方程        |
| • 计算统计内点(inliers) | 线到点的距离 $E$ |
| • 重复, 直到满足条件      | 95%        |





# 点云分割 拟合 聚类方法

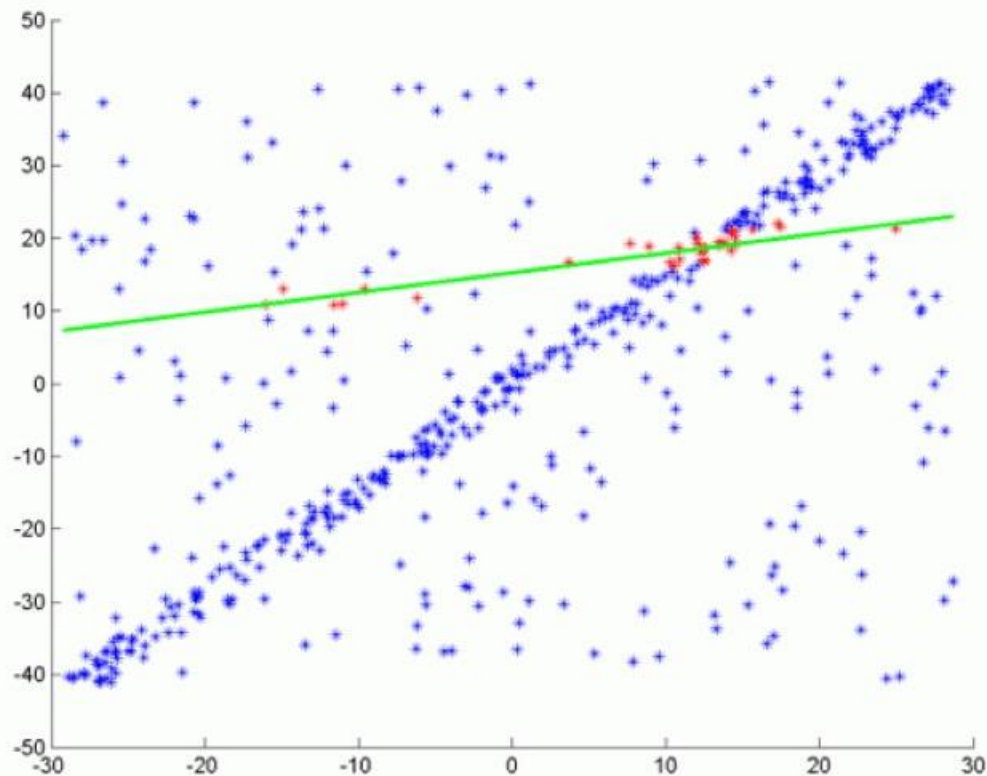
## ➤ 分割拟合

随机采样一致性:

RANSAC (Random Sample Consensus)

是一种鲁棒模型拟合的随机算法:

- |                   |            |
|-------------------|------------|
| • 选择样本集           | 线模型        |
| • 计算模型            | 2个点        |
| • 计算统计内点(inliers) | 线方程        |
| • 重复, 直到满足条件      | 线到点的距离 $E$ |
|                   | 95%        |



# 点云分割 拟合 聚类方法

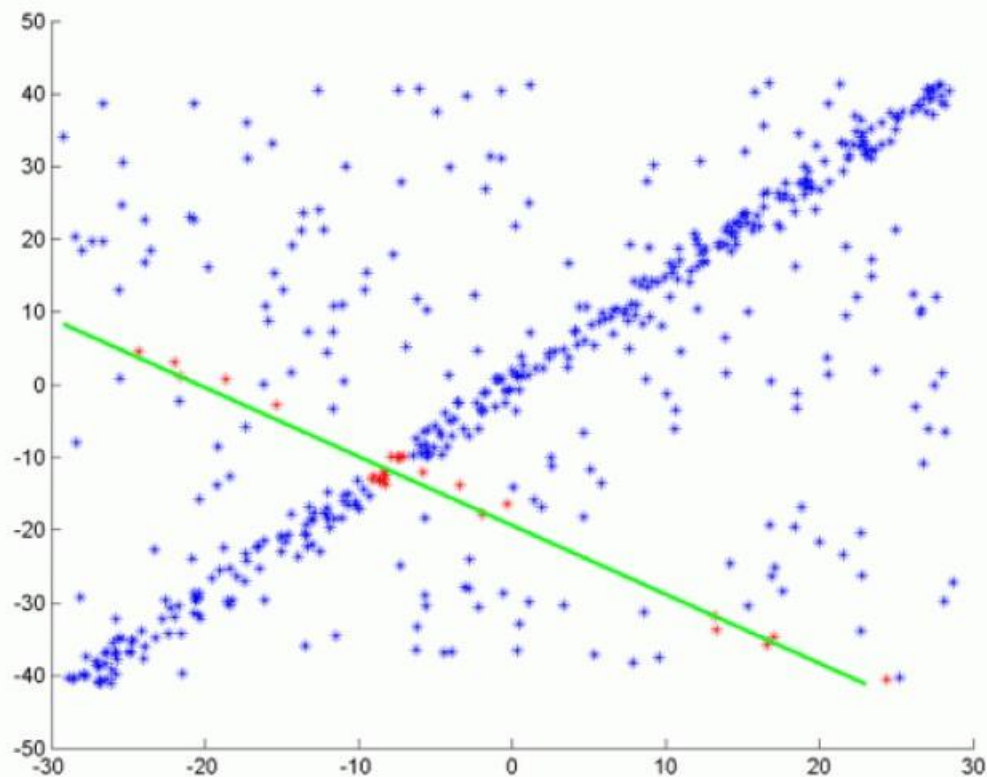
## ➤ 分割拟合

随机采样一致性:

RANSAC (Random Sample Consensus)

是一种鲁棒模型拟合的随机算法:

- |                   |            |
|-------------------|------------|
| • 选择样本集           | 线模型        |
| • 计算模型            | 2个点        |
| • 计算统计内点(inliers) | 线方程        |
| • 重复, 直到满足条件      | 线到点的距离 $E$ |
|                   | 95%        |



# 点云分割 拟合 聚类方法

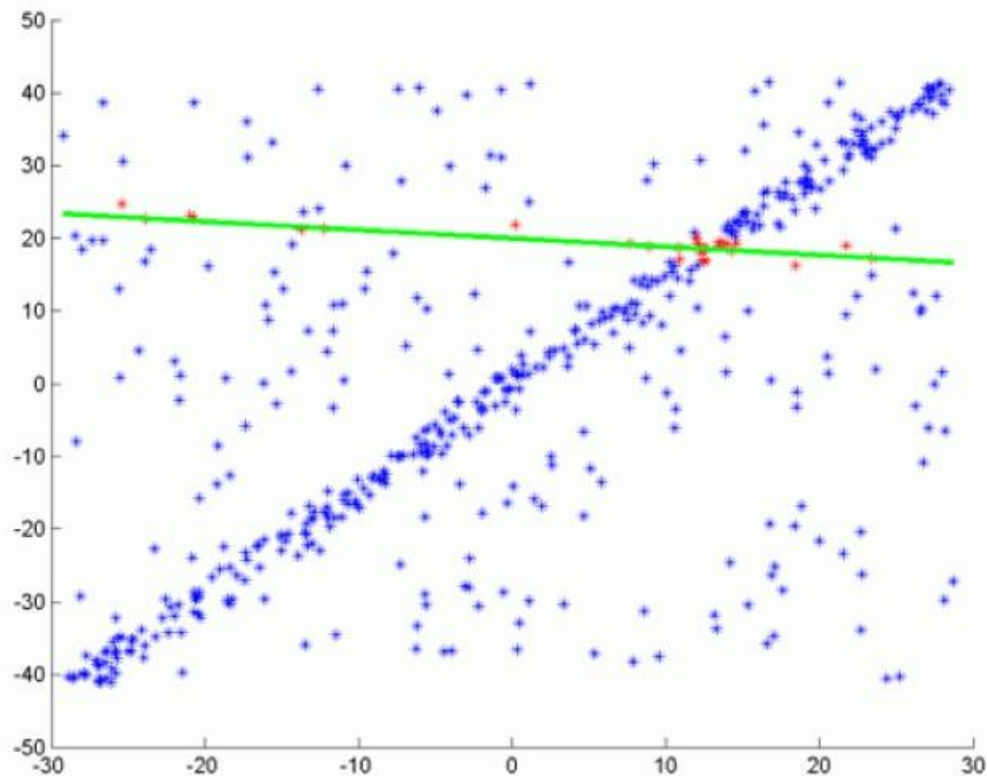
## ➤ 分割拟合

随机采样一致性:

RANSAC (Random Sample Consensus)

是一种鲁棒模型拟合的随机算法:

- |                   |            |
|-------------------|------------|
| • 选择样本集           | 线模型        |
| • 计算模型            | 2个点        |
| • 计算统计内点(inliers) | 线方程        |
| • 重复, 直到满足条件      | 线到点的距离 $E$ |
|                   | 95%        |





# 点云分割 拟合 聚类方法

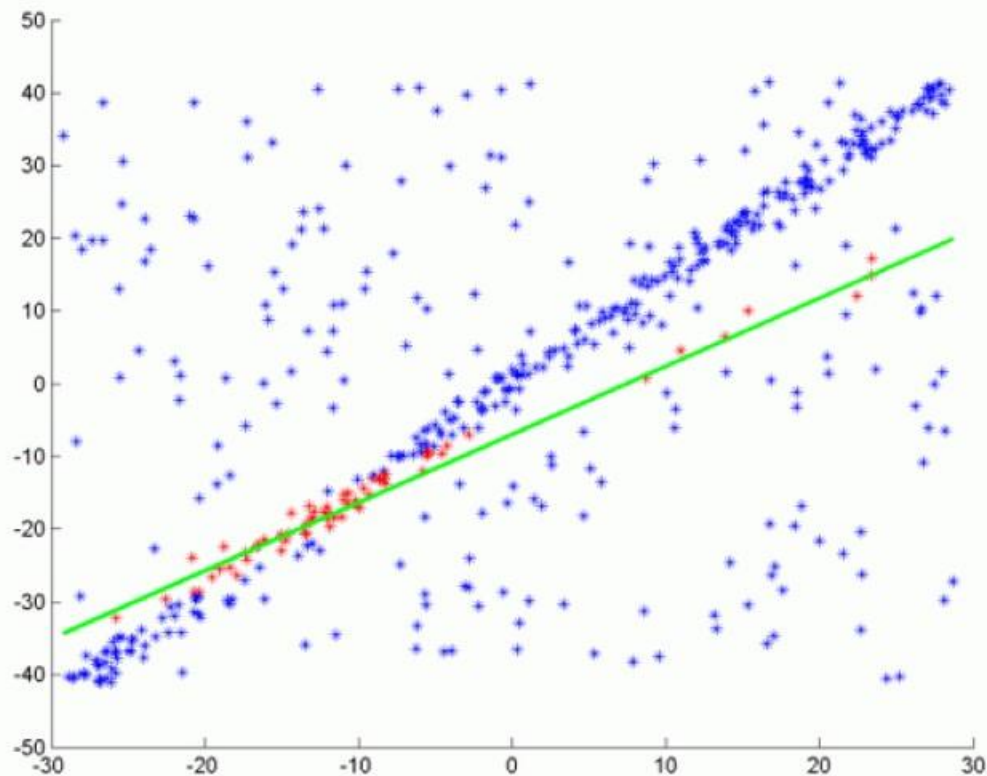
## ➤ 分割拟合

随机采样一致性:

RANSAC (Random Sample Consensus)

是一种鲁棒模型拟合的随机算法:

- |                   |            |
|-------------------|------------|
| • 选择样本集           | 线模型        |
| • 计算模型            | 2个点        |
| • 计算统计内点(inliers) | 线方程        |
| • 重复, 直到满足条件      | 线到点的距离 $E$ |
|                   | 95%        |



# 点云分割 拟合 聚类方法

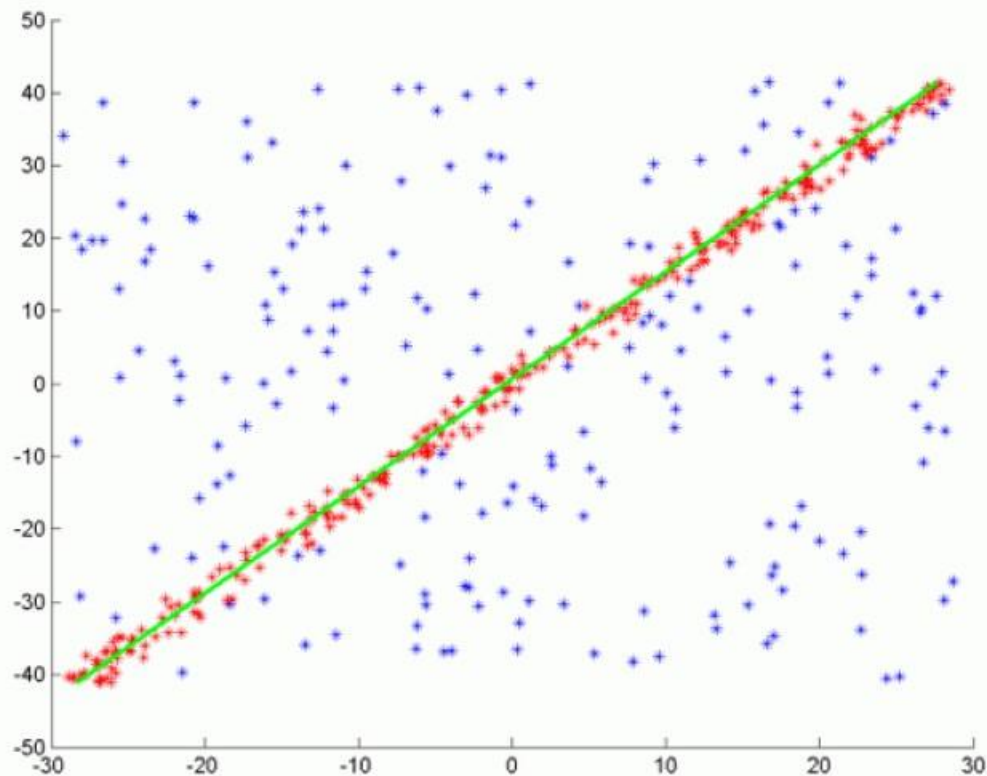
## ➤ 分割拟合

随机采样一致性:

RANSAC (Random Sample Consensus)

是一种鲁棒模型拟合的随机算法:

- |                   |            |
|-------------------|------------|
| • 选择样本集           | 线模型        |
| • 计算模型            | 2个点        |
| • 计算统计内点(inliers) | 线方程        |
| • 重复, 直到满足条件      | 线到点的距离 $E$ |
|                   | 95%        |



# 点云分割 拟合 聚类方法

## ➤ 分割拟合 pcl

采样一致性算法:

- 随机采样一致性算法 (RANSAC)
- 加权采样一致性算法 (MSAC)
- 最大似然一致性算法 (MLESC)
- 最小中值方差一致性算法 (LMEDS)
- 渐进采样一致性算法 (PROSAC)

```
pcl::SampleConsensusModel< PointT >
->pcl::SampleConsensusModelPlane< PointT >
->pcl::SampleConsensusModelLine< PointT >
->pcl::SampleConsensusModelCircle2D< PointT >
->pcl::SampleConsensusModelCircle3D< PointT >
->pcl::SampleConsensusModelSphere< PointT >
->pcl::SampleConsensusModelCone< PointT ,PointNT >
->pcl::SampleConsensusModelCylinder< PointT ,PointNT >
->pcl::SampleConsensusModelRegistration< PointT >
->pcl::SampleConsensusModelStick< PointT >
```

拟合模型类型:

1.平面模型

SACMODEL\_PLANE[normal\_x normal\_y normal\_z d]

2.线模型

SACMODEL\_LINE

[point\_on\_line.x point\_on\_line.y point\_on\_line.z  
line\_direction.x line\_direction.y line\_direction.z]

3.平面圆模型

SACMODEL\_CIRCLE2D [center.x center.y radius]

4.三维圆模型

SACMODEL\_CIRCLE3D

[center.x, center.y, center.z, radius, normal.x,  
normal.y, normal.z]

5.球模型

SACMODEL\_SPHERE

6.圆柱体模型

SACMODEL\_CYLINDER[point\_on\_axis.x point\_on\_axis.y  
point\_on\_axis.z axis\_direction.x axis\_direction.y  
axis\_direction.z radius]

7.圆锥体模型

SACMODEL\_CONE

[apex.x, apex.y, apex.z, axis\_direction.x,  
axis\_direction.y, axis\_direction.z, opening\_angle]



# 点云分割 拟合 聚类方法

## ➤ 分割拟合 pcl

采样一致性算法:

```
// necessary includes
#include <pcl/sample_consensus/ransac.h>
#include <pcl/sample_consensus/sac_model_plane.h>
// ...
// 创建平面模型采样一致性对象
SampleConsensusModelPlane<PointXYZ>::Ptr model
(new SampleConsensusModelPlane<PointXYZ> (input));
// 创建随机采样一致性算法对象
RandomSampleConsensus<PointXYZ> sac (model, 0.03);
// 计算分割
bool result = sac.computeModel ();
```

- 创建探测平面的SAC对象
- 创建参数E=3cm的RANSAC算法
- 计算最佳模型（完整RANSAC，已迭代完成）

```
// 模型系数
pcl::ModelCoefficients::Ptr coefficients (new
pcl::ModelCoefficients);
pcl::PointIndices::Ptr inliers (new
pcl::PointIndices); //内点索引
// pcl::PointIndices::Ptr outliers (new
pcl::PointIndices); //外点索引
// 创建一个点云分割对象
pcl::SACSegmentation<pcl::PointXYZ> seg;
// 是否优化模型系数
seg.setOptimizeCoefficients (true);
// 设置模型 和 采样方法
seg.setModelType (pcl::SACMODEL_PLANE); // 平面模型
seg.setMethodType (pcl::SAC_RANSAC); // 随机采样一致性算法
seg.setDistanceThreshold (0.01); //是否在平面上的阈值

seg.setInputCloud (cloud); //输入点云
seg.segment (*inliers, *coefficients); //分割 得到平面系数
已经在平面上的点的 索引
```

# 点云分割 拟合 聚类方法

## ➤ 分割拟合 pcl

采样一致性算法:

```
//获取内点索引
boost::shared_ptr<vector<int> > inliers (new vector<int>);
sac.getInliers (*inliers);
cout << "Found model with " << inliers->size () << " inliers";
//获取模型拟合参数
Eigen::VectorXf coeff; sac.getModelCoefficients (coeff);
cout << ", plane normal is: " << coeff[0] << ", " << coeff[1] << ", "
//重新拟合分割
Eigen::VectorXf coeff_refined;
model->optimizeModelCoefficients(*inliers, coeff, coeff_refined);
model->selectWithinDistance(coeff_refined, 0.03, *inliers);
//拟合投影
PointCloud<PointXYZ> proj_points;
model->projectPoints (*inliers, coeff_refined, proj_points);
```

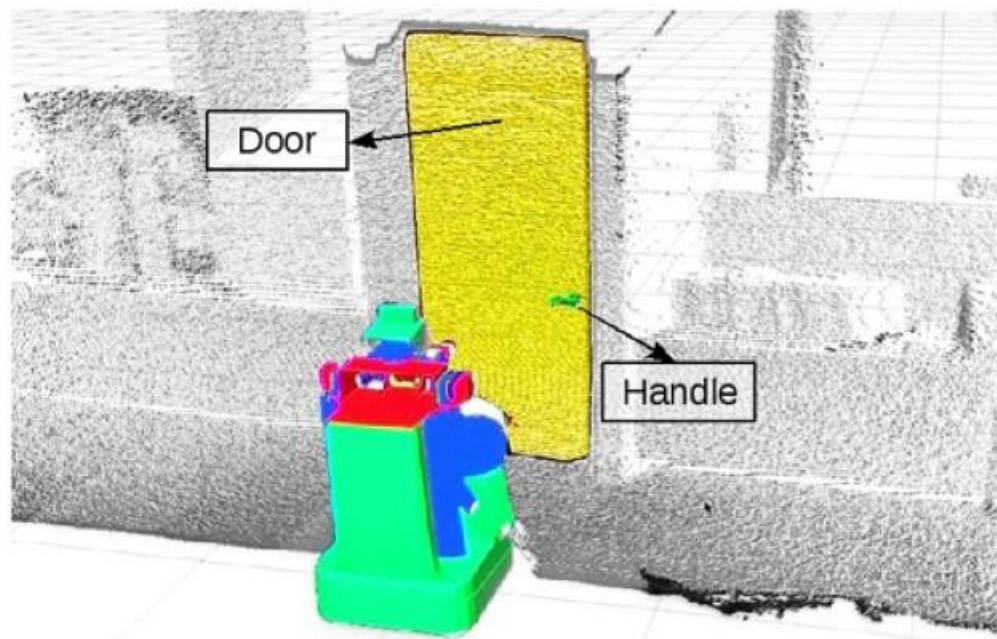
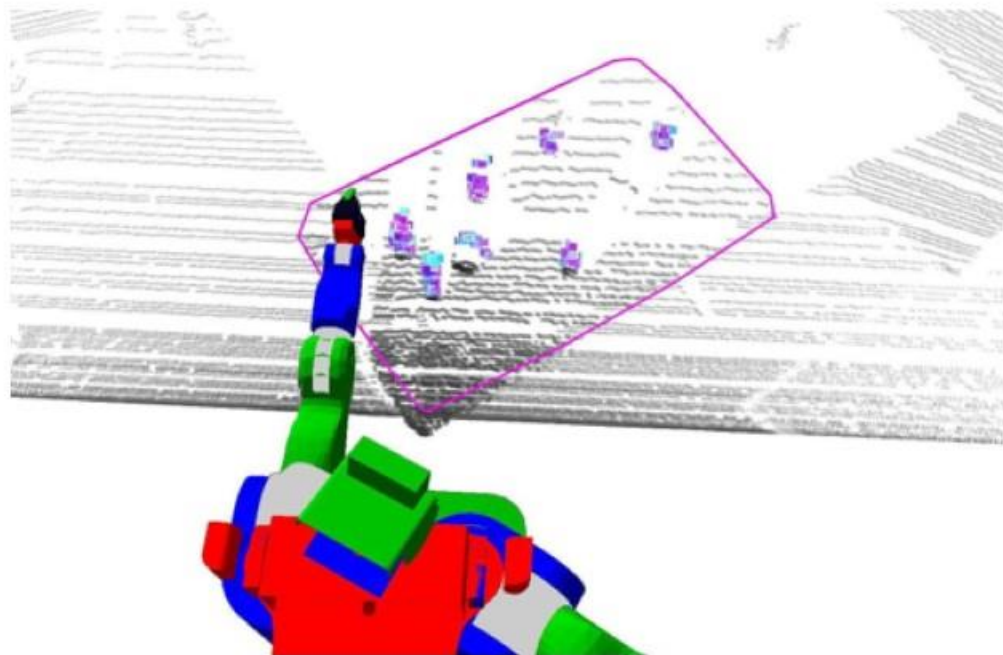
- 获得分割的**内点索引**
- 计算模型拟合的**参数**
- 优化拟合模型参数 (**refitting**)
- **投影**内点到分割的模型

```
// Set normals
model->setInputNormals(normals);
// Set the normal angular distance weight.
model->setNormalDistanceWeight(0.5f);
```



# 点云分割 拟合 聚类方法

## ➤ 分割拟合聚类



- 分割出平面模型
- 聚类找出桌子或门上的物体或者把手
- 计算平面的凸包
- 聚类分割物体

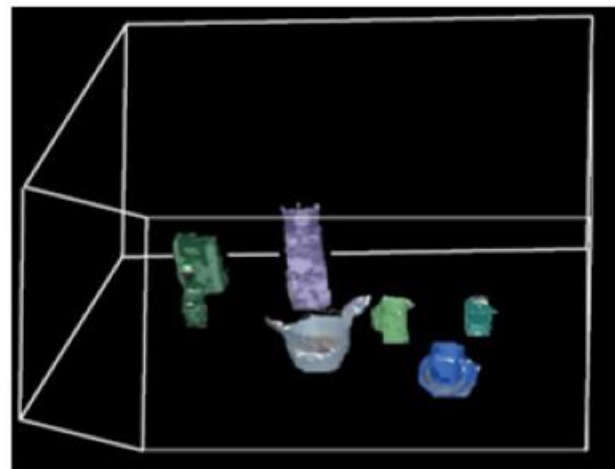
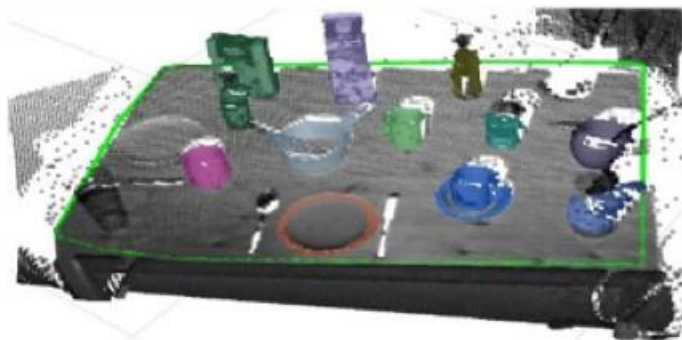


# 点云分割 拟合 聚类方法

## ➤ 分割拟合聚类

ExtractPolygonalPrismData

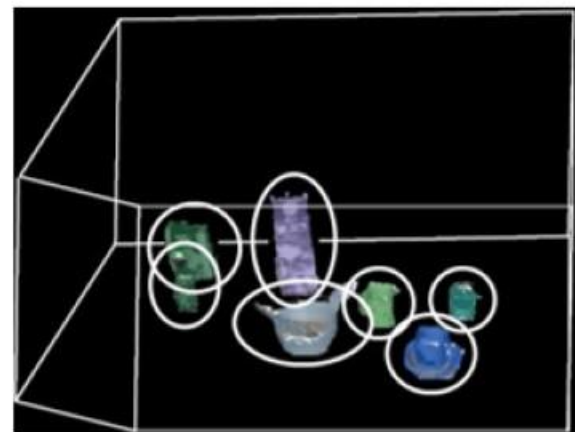
提取多边形棱柱



```
// 根据投影后的内点创建凸包 ConvexHull
pcl::PointCloud<pcl::PointXYZ>::Ptr cloud_hull
(new pcl::PointCloud<pcl::PointXYZ>);
pcl::ConvexHull<pcl::PointXYZ> chull;
chull.setInputCloud (inliers_cloud);
chull.reconstruct (*cloud_hull);
```

```
// 分割出在多边形棱柱内的点
ExtractPolygonalPrismData<PointXYZ> ex;
ex.setInputCloud (outliers);
ex.setInputPlanarHull (cloud_hull);
PointIndices::Ptr output (new PointIndices);
ex.segment (*output);
```

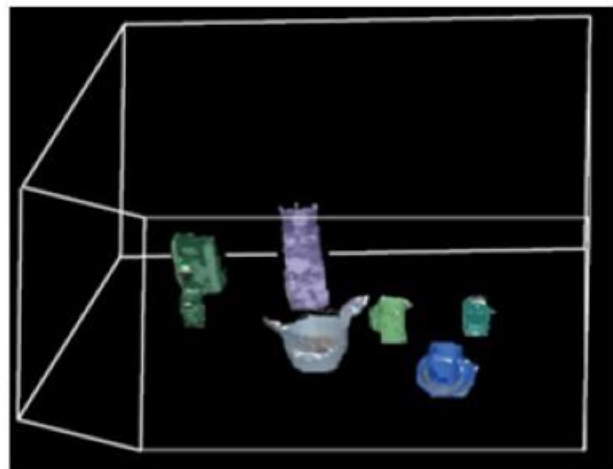
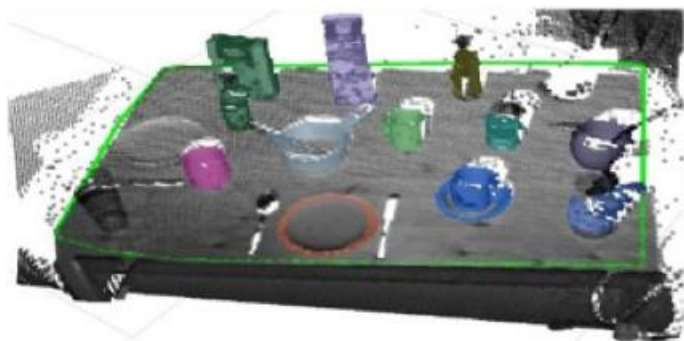
- 计算凸包
- 传递给ExtractPolygonalPrismData
- 然后 聚类!



# 点云分割 拟合 聚类方法

## ➤ 分割拟合聚类

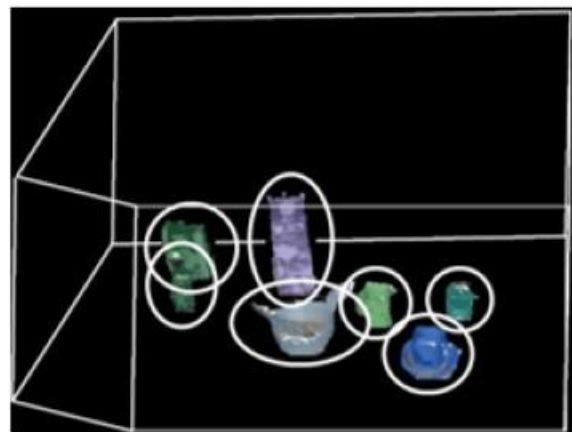
**EuclideanClusterExtraction**  
欧几里得聚类算法(区域生长)



```
// 创建EuclideanClusterExtraction并设置参数
pcl::EuclideanClusterExtraction<PointT> ec;
ec.setClusterTolerance (cluster_tolerance);
ec.setMinClusterSize (min_cluster_size);
ec.setMaxClusterSize (max_cluster_size);
ec.setSearchMethod (tree);
```

```
// 设置输入点云并计算
ec.setInputCloud (input);
ec.extract (cluster_indices_out);
```

- 计算凸包
- 传递给**ExtractPolygonalPrismData**
- 然后 聚类!



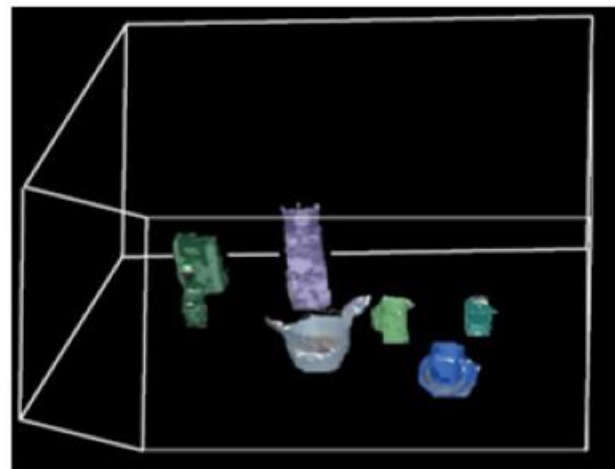
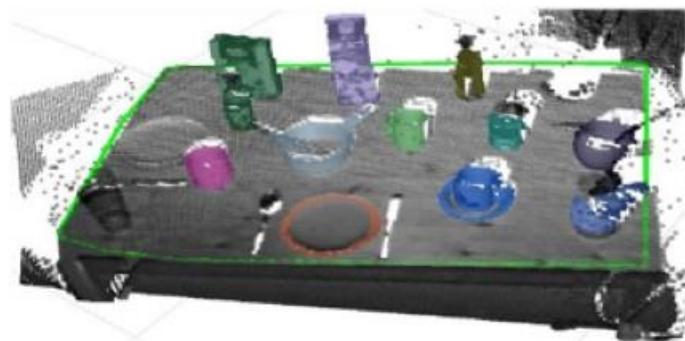


# 点云分割 拟合 聚类方法

## ➤ 分割拟合聚类

### EuclideanClusterExtraction

欧几里得聚类算法 (区域生长)

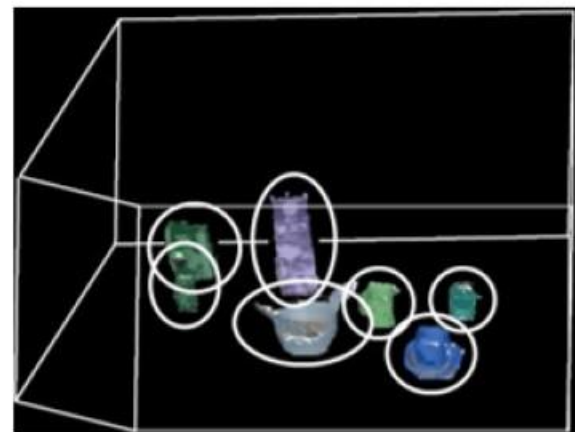


具体的实现方法（原理是将一个点云团聚合成一类）：

- 1. 找到空间中某点 $p_{10}$ ，用kdTree找到离他最近的 $n$ 个点，判断这 $n$ 个点到 $p_{10}$ 的距离。
- 将距离小于阈值 $r$ 的点 $p_{12}, p_{13}, p_{14}, \dots$ 放在类 $Q$ 里
- 2. 在  $Q \setminus p_{10}$  里找到一点 $p_{12}$ , 重复1
- 3. 在  $Q \setminus p_{10}, p_{12}$  找到一点，重复1，找到 $p_{22}, p_{23}, p_{24}, \dots$ 全部放进 $Q$ 里
- 4. 当  $Q$  再也不能有新点加入了，则完成搜索了

条件欧几里得聚类法：

- **ConditionalEuclideanClustering**
- `clustering.setConditionFunction(&customCondition);`





# 点云分割 拟合 聚类方法

## ➤ 分割拟合聚类

### RegionGrowing

区域生长算法（法线差值与曲率差值）

将具有相似性的像素集合起来构成区域

具体的实现方法：

- 0. 计算 法线normal 和 曲率curvatures，依据曲率升序排序；
- 1. 选择曲率最低的为初始种子点，种子周围的临近点和种子点云相比较；
- 2. 法线的方向是否足够相近（法线夹角足够平滑），法线夹角阈值；
- 3. 曲率是否足够小(表面处在同一个弯曲程度)，曲率差值阈值；
- 4. 如果满足2，3则该点可用做种子点；
- 5. 如果只满足2，则归类而不做种子；

条件欧几里得聚类法：

- **ConditionalEuclideanClustering**
- `clustering.setConditionFunction(&customCondition);`

# 点云分割 拟合 聚类方法

## ➤ 分割拟合聚类

### RegionGrowing

区域生长算法（法线差值与曲率差值）

//区域增长聚类分割对象 <点, 法线>

```
pcl::RegionGrowing<pcl::PointXYZ, pcl::Normal> reg;
```

```
reg.setMinClusterSize (50); //最小的聚类的点数
```

```
reg.setMaxClusterSize (1000000); //最大的聚类的点数
```

```
reg.setSearchMethod (tree); //搜索方式
```

```
reg.setNumberOfNeighbours (30); //设置搜索的邻域点的个数
```

```
reg.setInputCloud (cloud); //输入点
```

```
//reg.setIndices (indices);
```

```
reg.setInputNormals (normals); //输入的法线
```

```
reg.setSmoothnessThreshold (3.0 / 180.0 * M_PI); //设置平滑度 法线差值阈值
```

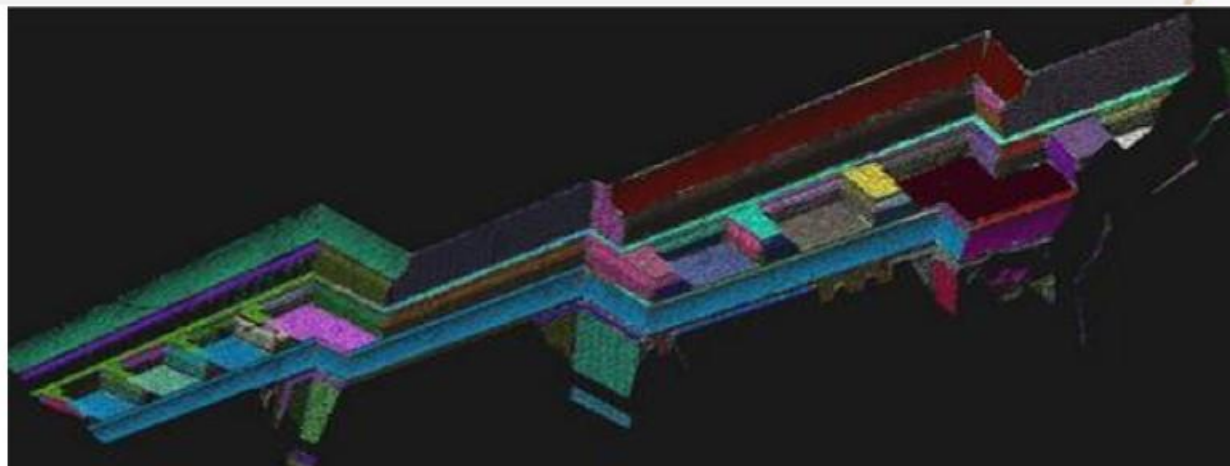
```
reg.setCurvatureThreshold (1.0); //设置曲率的阈值
```

```
std::vector<pcl::PointIndices> clusters;
```

```
reg.extract (clusters); //提取点的索引
```

基于颜色的区域生长算法:

- RegionGrowingRGB





# 内容概要

点云滤波方法

点云组织形式与最近邻搜索

点云分割 拟合 聚类方法

激光雷达运动补偿方法

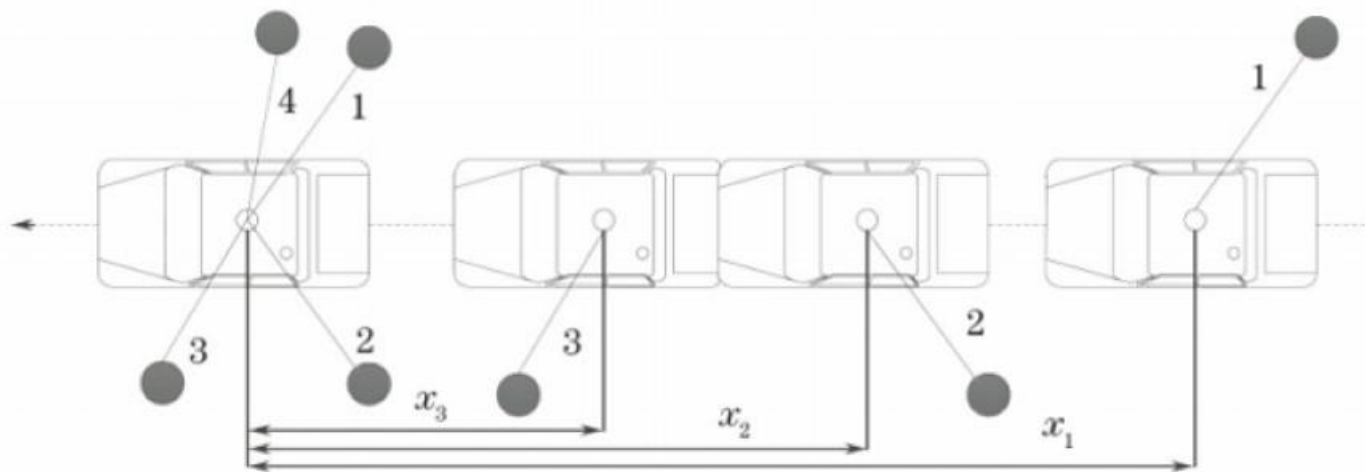
点云去地面原理及方法



# 激光雷达运动补偿方法

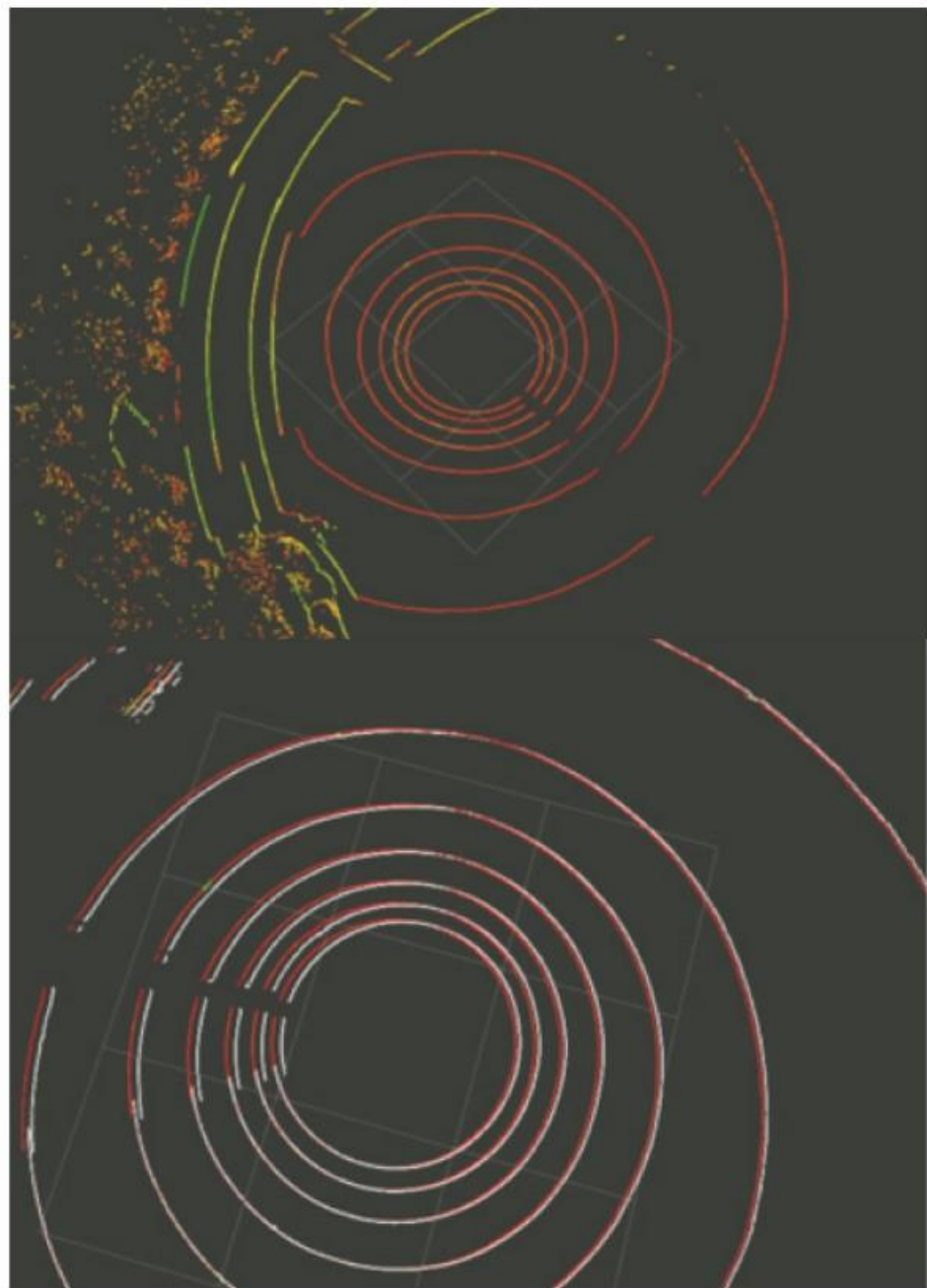
## ➤ 运动补偿

运动状态估计与插值计算



Apollo激光雷达感知:

- [apollo/modules/drivers/velodyne/compensator/](#)





# 内容概要

点云滤波方法

点云组织形式与最近邻搜索

点云分割 拟合 聚类方法

激光雷达运动补偿方法

点云去地面原理及方法

# 点云去地面原理及方法

## ➤ 去地面原理分类

原理分类:

- 基于平面栅格的方法
- 基于模型拟合的方法
- 基于面元mesh的方法
- .....

基于平面栅格的方法:

- 密度
- 平均高度
- 高度方差
- 最大高度差
- 相对高度

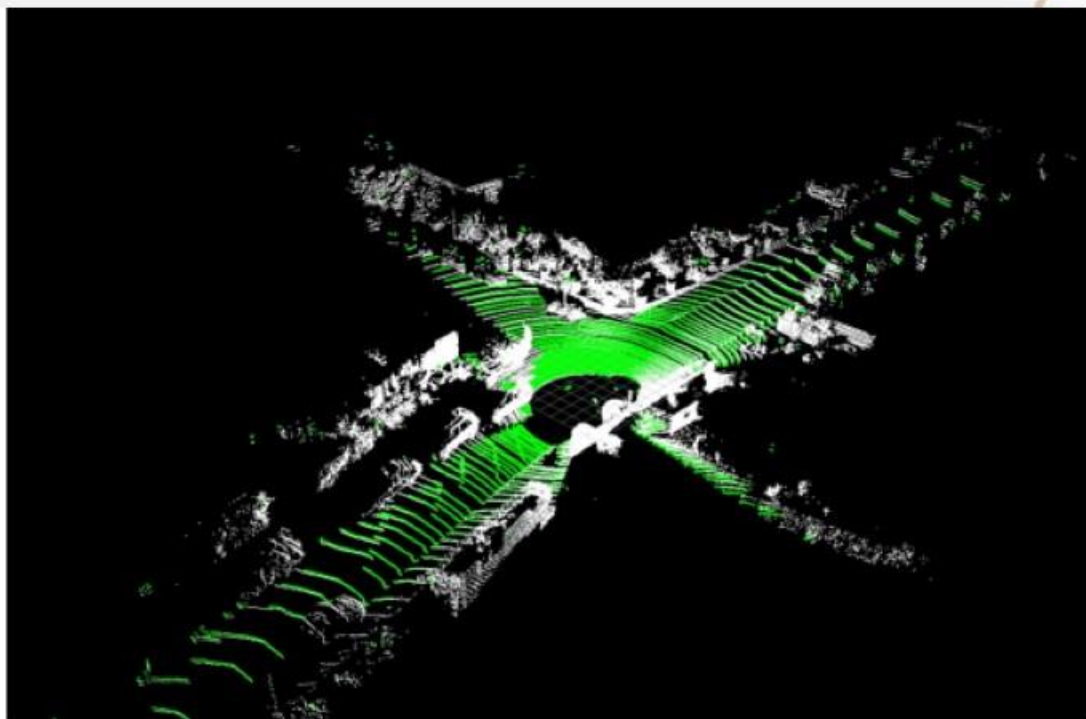
基于模型拟合的方法:

- (分段)平面拟合-RANSAC
- 高斯过程-GP-INSAC
- 平面参数拟合(特征值)

基于面元mesh的方法:

- 构建mesh, 区域生长

1. Douillard, B. , et al. "Hybrid elevation maps: 3D surface models for segmentation." *IEEE/RSJ International Conference on Intelligent Robots & Systems* IEEE, 2010.
2. Himmelsbach, M. , F. V. Hundelshausen , and H. J. Wuensche . "Fast Segmentation of 3D Point Clouds for Ground Vehicles." *IEEE Intelligent Vehicles Symposium* IEEE, 2010.
3. Zermas, Dimitris , I. Izzat , and N. Papanikolopoulos . "Fast Segmentation of 3D Point Clouds: A Paradigm on LiDAR Data for Autonomous Vehicle Applications." *International Conference on Robotics and Automation* IEEE, 2017.





# 点云去地面原理及方法

## ➤ 去地面原理方法

### 平面参数拟合(特征值):

- 三维平面模型

平面几何模型:

$$\begin{cases} ax + by + cz = d \\ \mathbf{n}^T \mathbf{x} = d \end{cases}$$

$$a\bar{x} + b\bar{y} + c\bar{z} = d$$

$$A\mathbf{n} = \begin{bmatrix} x_1 - \bar{x} & y_1 - \bar{y} & z_1 - \bar{z} \\ x_2 - \bar{x} & y_2 - \bar{y} & z_2 - \bar{z} \\ \dots & \dots & \dots \\ x_n - \bar{x} & y_n - \bar{y} & z_n - \bar{z} \end{bmatrix} \begin{bmatrix} a \\ b \\ c \end{bmatrix} = 0$$

目标函数为:

$$\mathbf{n}^* = \operatorname{argmin}_{\mathbf{n}} \|\mathbf{A}\mathbf{n}\|$$
$$\|\mathbf{n}\| = 1$$

对A矩阵作奇异值分解 (SVD)  
D是对角矩阵,  
U和V均为酉矩阵:

$$\begin{cases} \|\mathbf{A}\mathbf{n}\| = \|\mathbf{U}\mathbf{D}\mathbf{V}^T \mathbf{n}\| = \|\mathbf{D}\mathbf{V}^T \mathbf{n}\| \\ \|\mathbf{V}^T \mathbf{n}\| = \|\mathbf{n}\| = 1 \end{cases}$$

最后一个对角元素为最小奇异值,  
 $\|\mathbf{A}\mathbf{n}\|$ 可取得最小值。  
即:

$$\mathbf{V}^T \mathbf{n} = \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix} \Rightarrow \mathbf{n}^* = \mathbf{V} \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix}$$

目标函数 $\min\|\mathbf{A}\mathbf{x}\|$ 的最优解为:

$$\mathbf{n}^* = \begin{bmatrix} v_1 & v_2 & v_3 \end{bmatrix} \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix} = v_3$$
$$\Rightarrow (a, b, c) = (\mathbf{n}^*)^T = (v_{3,1}, v_{3,2}, v_{3,3})$$

# 点云去地面原理及方法

## 去地面原理方法

平面参数拟合(特征值):

- 三维平面模型

